



School of Artificial Intelligence
Jilin University

第八章 分支限界法

(部 分 内 容 参 考 计 算 机 学 院
《 算 法 设 计 与 分 析 》)

吉林大学人工智能学院

王元宏

- 8.1 分支-限界法之检索方法
- 8.2 LC-检索
- 8.3 15-谜问题
- 8.4 再论LC-检索
- 8.5 0/1背包问题
- 8.6 分支-限界法之限界方法
- 8.7 小结

- 8.1 分支-限界法之检索方法
- 8.2 LC-检索
- 8.3 15-谜问题
- 8.4 再论LC-检索
- 8.5 0/1背包问题
- 8.6 分支-限界法之限界方法
- 8.7 小结

- 与回溯法相同点：
 - 分支限界法也是一种在问题的解空间树上搜索问题解的算法
 - 同样适用于求解组合数较大的问题，特别是组合优化问题(求最优解)
- 求解目标不完全相同
 - 回溯法的求解目标，是找出解空间树中满足约束条件的所有解或一个解
 - 分支限界法的求解目标，是在满足约束条件的解中找出使得某一目标函数达到极大或极小的解，即某种意义下的最优解

- 回溯法的改进
 - 根据树的分支情况设计优先策略，如优先搜索结点少/解多的分支
- 回溯法构造状态空间树
 - 当前的E-结点R一旦生成一个新的孩子结点C, 这个C结点就变成一个新的E-结点, 当检测完了子树C后, R结点就再次成为E-结点, 生成下一个孩子结点
 - 该方法也称为深度优先生成法，对应回溯法
- 第二种构造方法:
 - 一个E-结点一直保持到变成死结点为止，对应分枝限界法

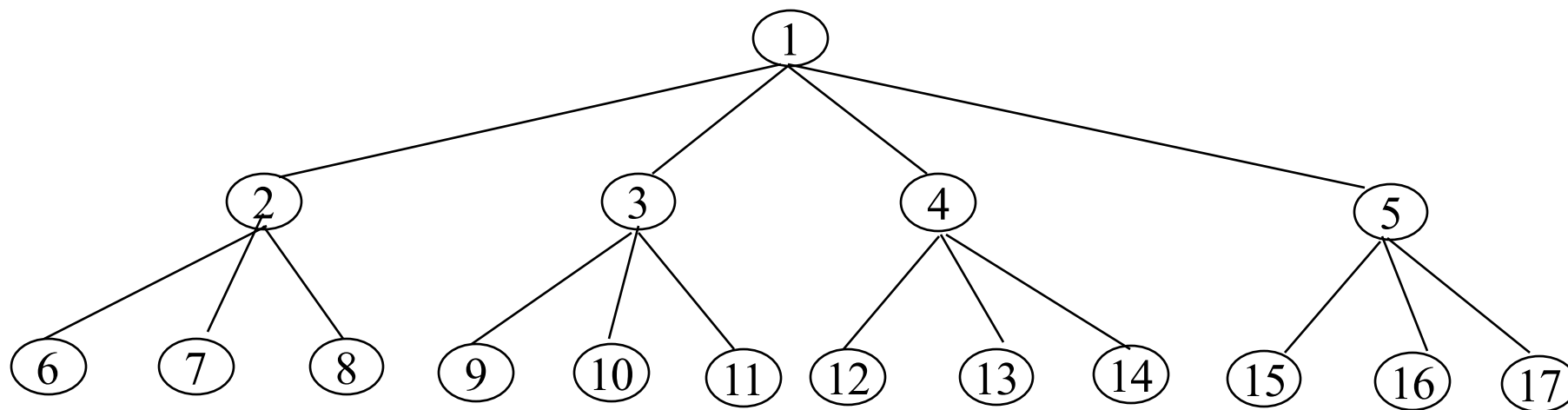
思考：如何改变搜索顺序？

- 初始时，解空间树的根节点被生成，根节点是唯一的活结点
- 按照**某种规则**，从活结点表中选取一个结点作为E-结点
- 生成E的所有儿子结点，判断每个子结点X：
 - 如果X是答案结点，输出从X到T的路径，算法结束；
 - 检验X，如果满足限界函数B，则将X添加到活结点表中；否则舍弃X
- 从活结点表中按照**某种规则**选择下一个结点成为新的E结点，重复上述操作
- 如果活结点表为空，则算法以失败结束

通过设计E-结点选取规则改变搜索顺序

- 根据活结点检索次序，分枝限界策略可以分为：
 - FIFO检索：活结点采用**先进先出**的方式(队列)出活结点表

FIFO检索四皇后问题的解空间树



活结点表

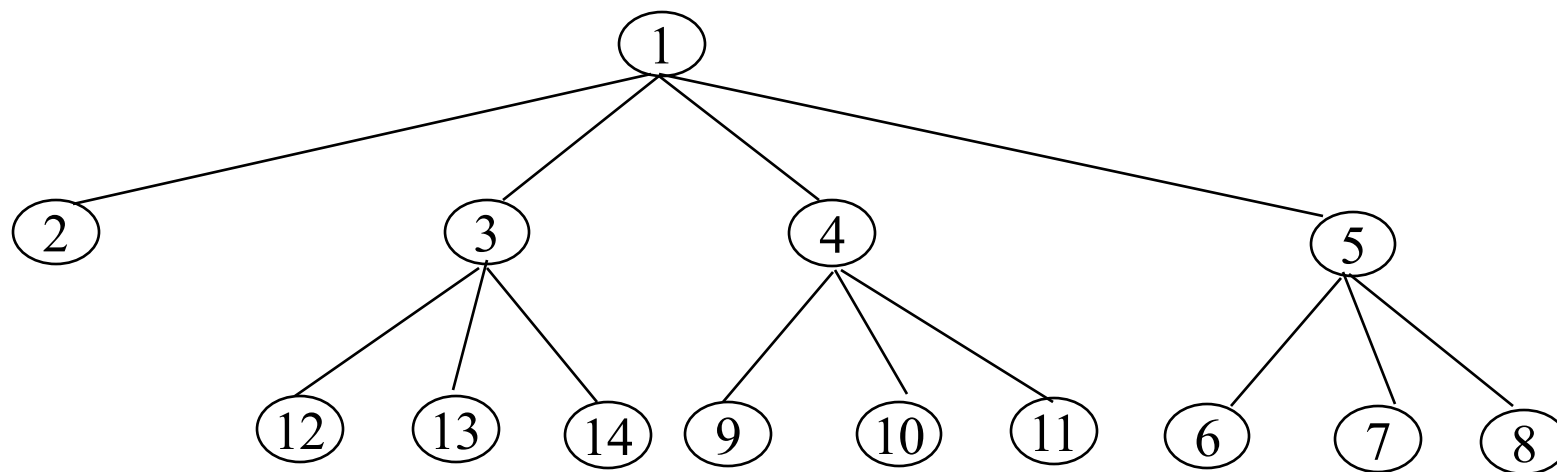
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	
--	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	--

- 根据活结点检索次序，分枝限界策略可以分为：
 - FIFO检索：活结点采用**先进先出**的方式(队列)出活结点表
 - LIFO检索：活结点采用**后进先出**的方式(栈)出活结点表

10

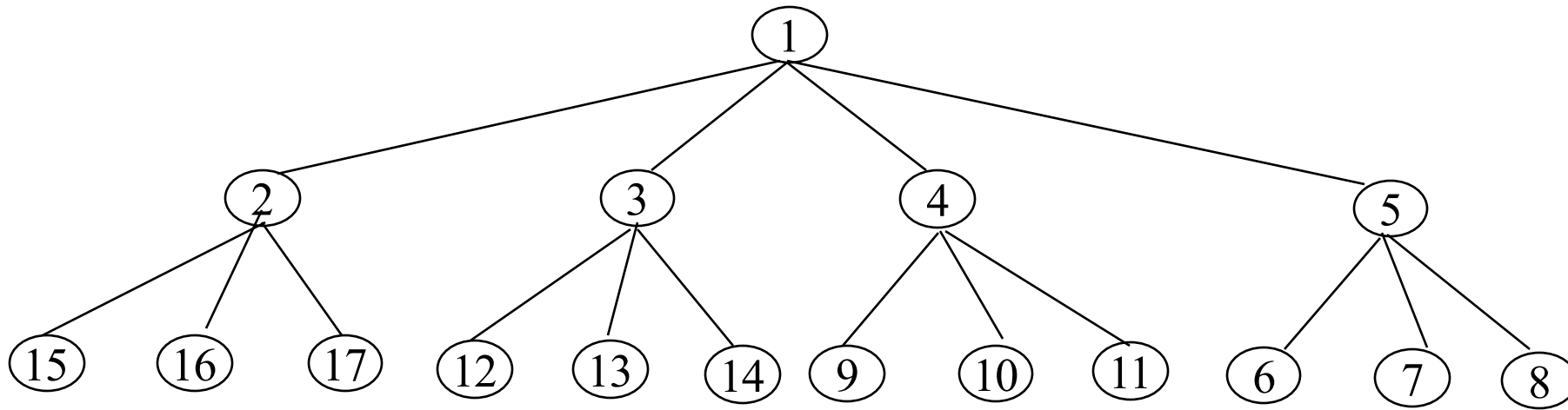
11

12

[illegible]

14

LIFO检索四皇后问题的解空间树



活结点表

procedure BB(T)

If T是答案结点 then 输出T; return end

找一个答案结点

E←T

将活结点表初始化为空

loop

for E的每个儿子X do

if X是答案结点 then 输出从X到T的那条路径; return end

if $B(X)$ then $ADD(X)$; PARENT(X)←E end

end

if 不再有活结点 then print(“no answer node”); return end

$LEAST(E)$

end

end

$B(X)$: 限界函数

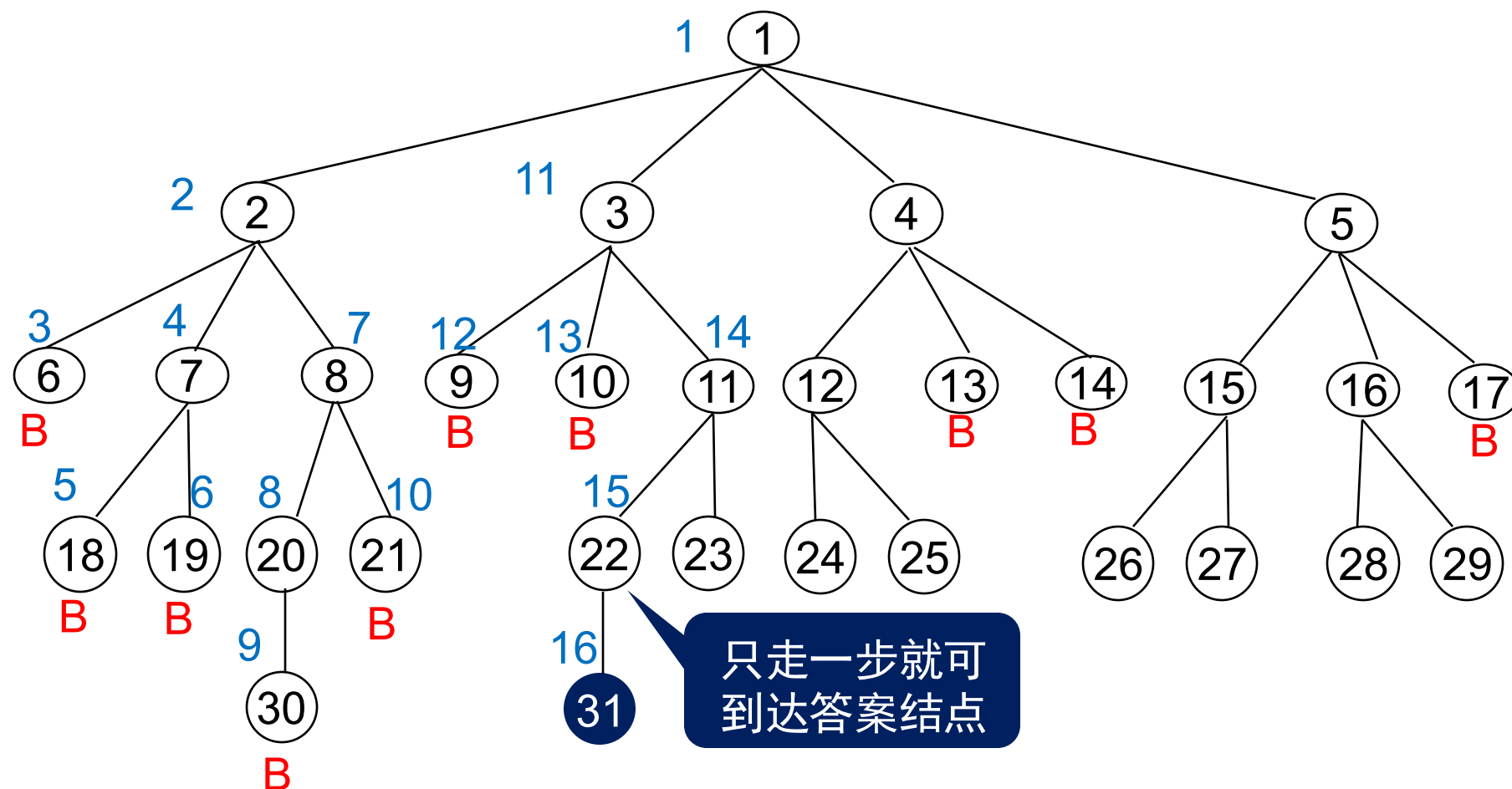
$ADD(X)$: 将X添加到活结点表中

$LEAST(E)$: 从活结点表中选中一个结点赋值给E, 并从表中删除该结点

思考：如果想获得所有可行解（答案结点），算法怎样设计？



FIFO-分枝限界法一共处理了31个结点，回溯法一共处理了16个结点



能否根据某个优先级，使得结点22和结点31比活结点表中其他活结点具有更高优先级，从而被快速得到？

- 在LIFO和FIFO分枝限界法中，对下一个E-结点的选择是死板的、盲目的，对于可能快速检索到答案结点的结点没有给出任何优先权
- 理想状态下，对活结点表使用一个“有智力的” **成本函数** c 来选取下一个E-结点，从而加快到达答案结点的检索速度

- 根据活结点检索次序，分枝限界策略可以分为：
 - FIFO检索：活结点采用**先进先出**的方式(队列)出活结点表
 - LIFO检索：活结点采用**后进先出**的方式(栈)出活结点表
 - LC检索(Least Cost search)：定义成本估计函数 $\hat{c}$ ，令**最小**函数值的活结点出活结点表

成本函数 c 的估计

思考：此处为何使用**估计**函数？

- 8.1 分支-限界法之检索方法
- 8.2 LC-检索
- 8.3 15-谜问题
- 8.4 再论LC-检索
- 8.5 0/1背包问题
- 8.6 分支-限界法之限界方法
- 8.7 小结

使用一个“有智力的”成本函数c来选取下一个E-结点

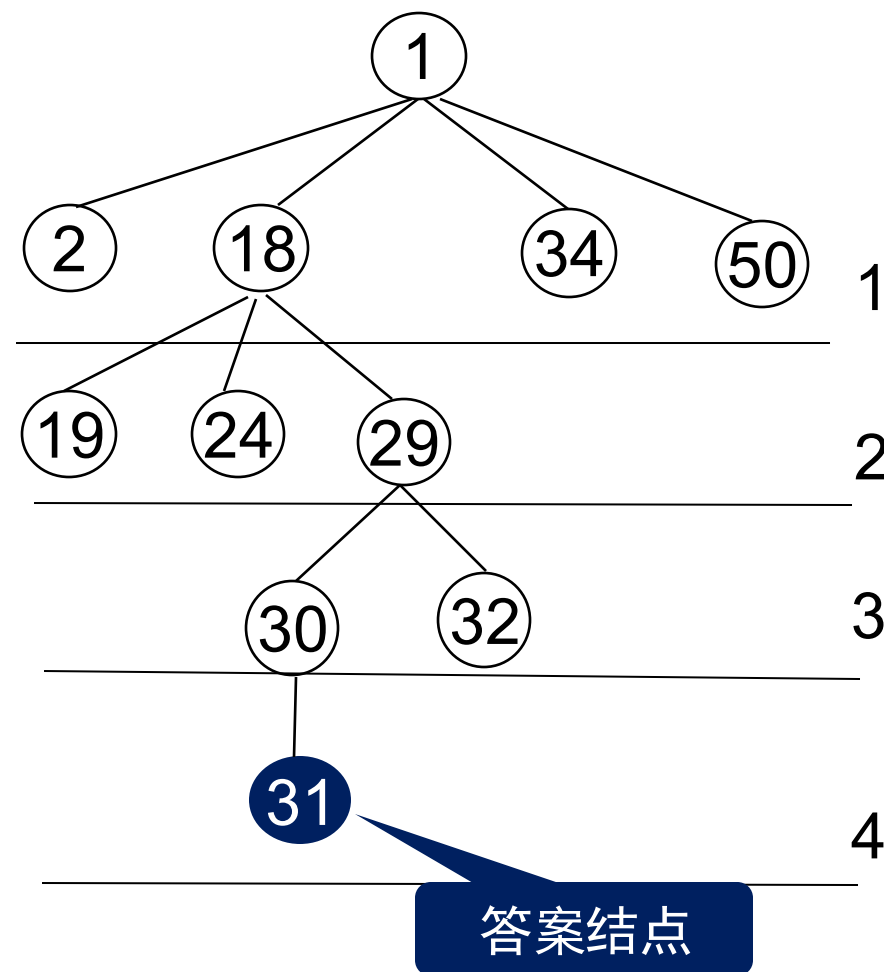
- $c(X)$ 的定义:
 - 方法1：基于X在生成一个答案结点之前需要生成的结点数
 - 寻找生成最小数目的答案结点
 - 方法2：基于距离X最近的那个答案结点的路径长度
 - 寻找路径长度最短的答案结点

选择后一种度量
来考虑问题

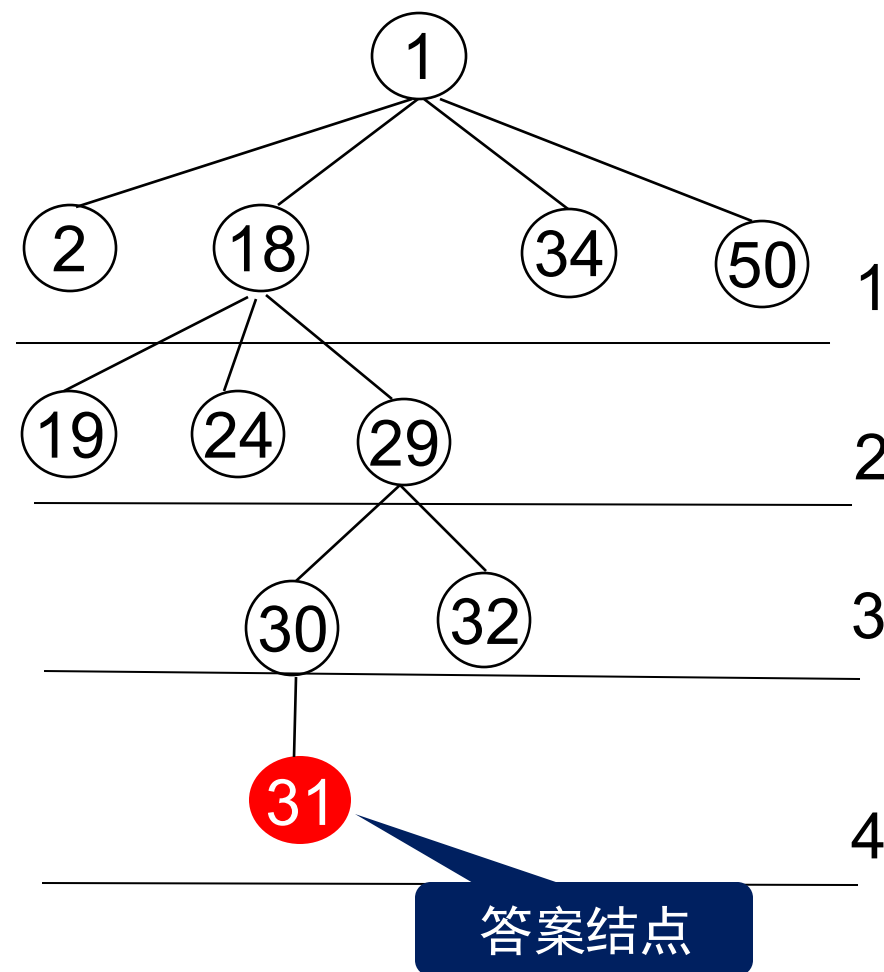
- 对于一个状态空间树里面的任何一个结点X，有如下几种可能：
 - X是答案结点；
 - X不是答案结点，但子树X中包含答案结点；
 - X不是答案结点，且子树X不包含任何答案结点；
- 从上帝视角定义一个结点成本函数 $c(X)$

- 如果X是答案结点
 - $c(X)$ =状态空间树的根结点到X的成本(即所用的代价, 可以是级数、计算复杂度等)
- 如果X不是答案结点且子树X不包含任何答案结点
 - 则 $c(X)=\infty$
- 如果X不是答案结点, 但子树X中包含答案结点
 - $c(X)$ =子树X中具有最小成本的答案结点的成本

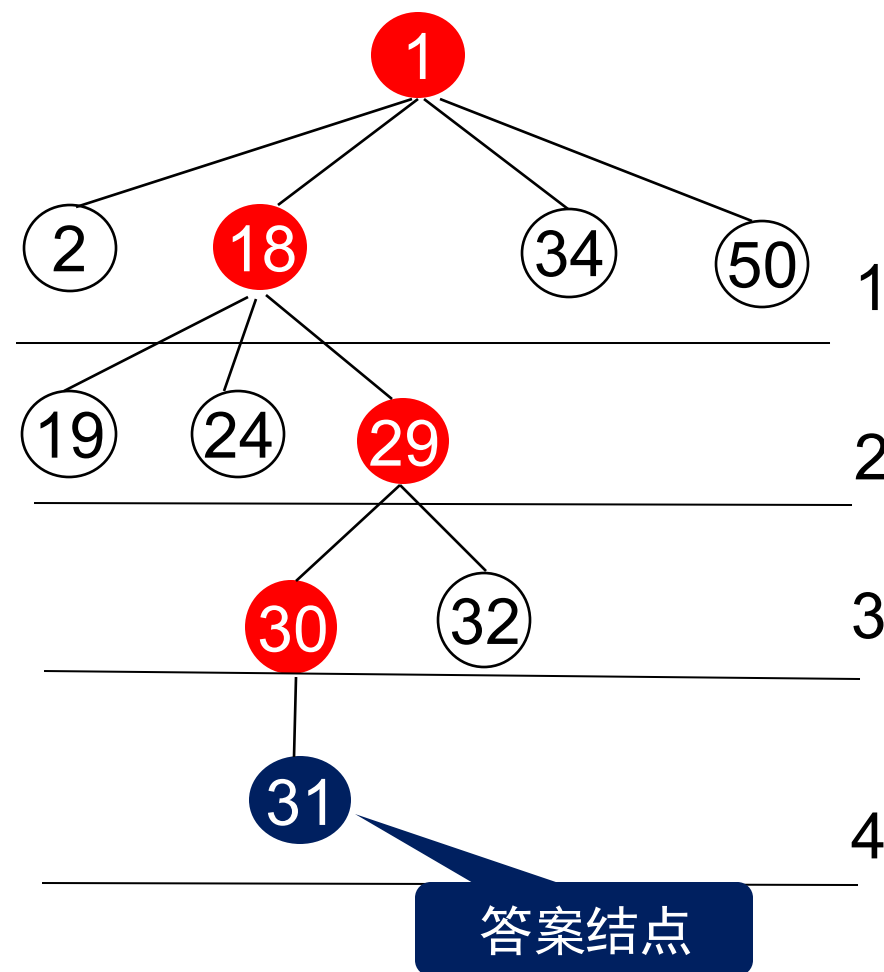
- 用级数来表达结点成本



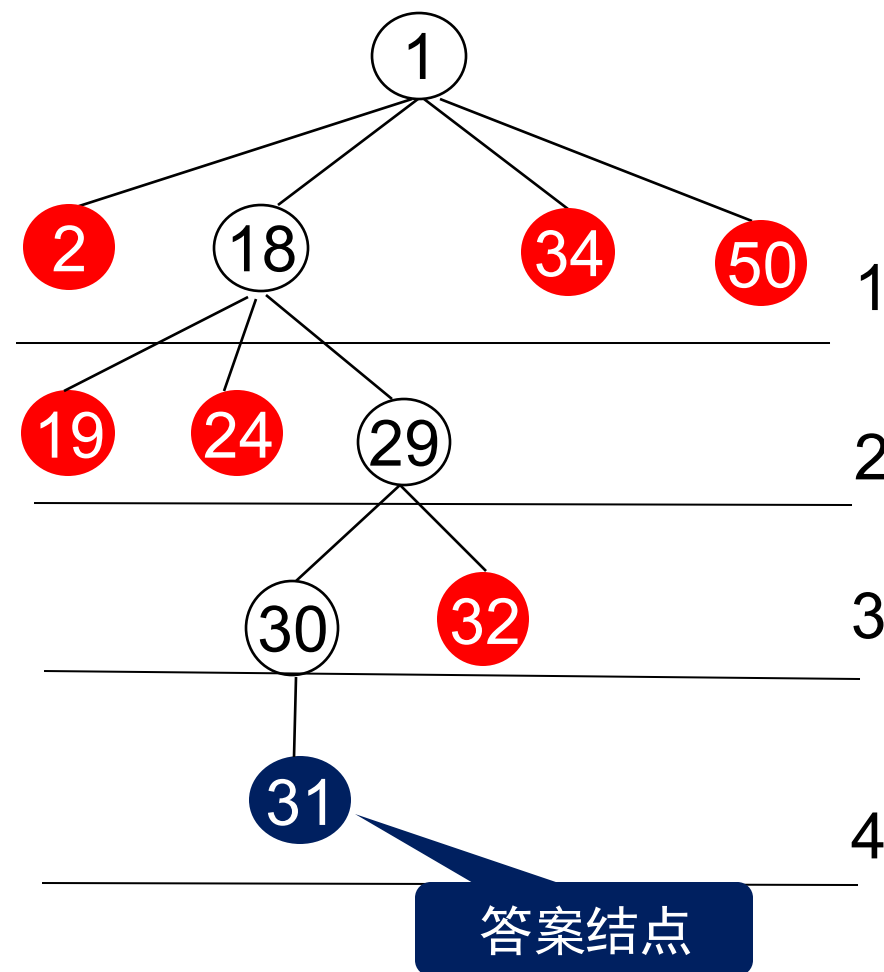
- 用级数来表达结点成本
- X是答案结点：
 - $c(31)=4$ （根结点到31的成本）



- 用级数来表达结点成本
- X是答案结点：
 - $c(31)=4$ （根结点到31的成本）
- X不是答案结点，但子树X中包含答案结点：
 - $c(1)=c(18)=c(29)=c(30)$
 $=c(31)$
 $=4$



- 用级数来表达结点成本
- X是答案结点：
 - $c(31)=4$ （根结点到31的成本）
- X不是答案结点，但子树X中包含答案结点：
 - $c(1)=c(18)=c(29)=c(30)$
 $=c(31)$
 $=4$
- X不是答案结点且子树X不包含任何答案结点：
 - $c(2)=\infty, c(19)=\infty, \dots$

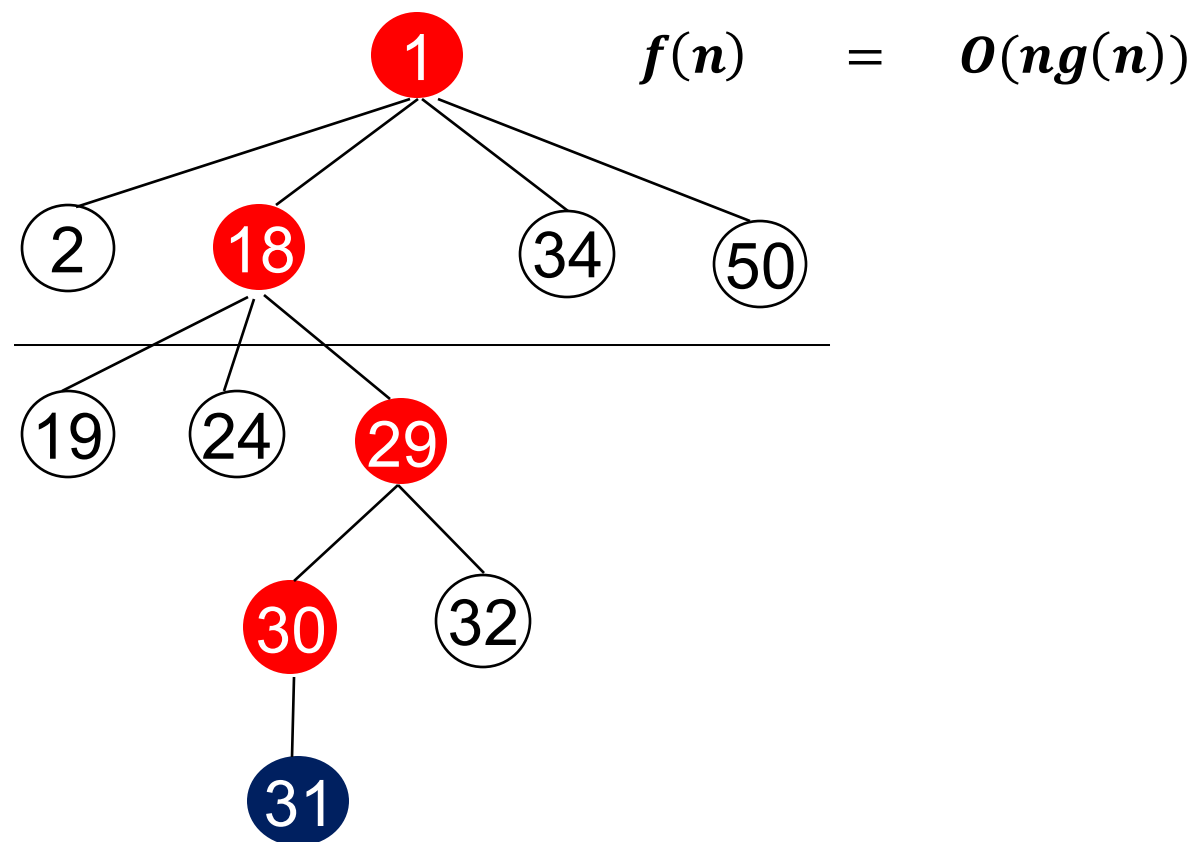


成本函数c的计算工作量

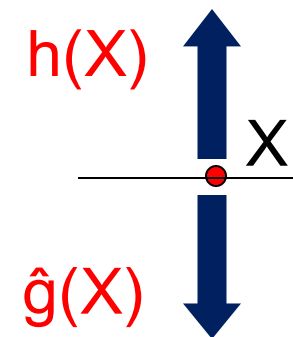
- 从上帝视角定义一个结点成本函数 $c(X)$ 为什么?
- c 的计算工作量与原问题具有类似的复杂度, 要得到函数 c 并不现实
- 一般情况只能大致估计

LEAST(E)每次选取 c 不等于 ∞ 的结点
最多 n 步即可求解原问题

计算 c



- 考虑结点X到一个答案结点的估计成本 $\hat{g}(X)$
 - 用来估计由X到达一个答案结点所需做的附加工作量
- 考虑根结点到结点X的成本 $h(X)$
- 为调整 h 和 $\hat{g}$ 在成本估计函数 $\hat{c}$ 中的影响比例，定义一个非负单调非递减函数 f
 - 调整 h 和 $\hat{g}$ 在成本估计函数 $\hat{c}$ 中的影响比例
- 成本估计函数： $\hat{c}(X)=f(h(X))+\hat{g}(X)$



利用 $\hat{c}$ 对活结点表进行检索时，优先选择更靠近答案结点但又离根结点较近的结点

思考：LC-分支限界法一定能找到答案结点吗？

procedure BB(T)

 If T是答案结点 then 输出T; return end

$E \leftarrow T$

 将活结点表初始化为空

 loop

 for E的每个儿子X do

 if X是答案结点 then 输出从X到T的那条路径; return end

 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end

 end

 if 不再有活结点 then print(“no answer node”); return end

LEAST(E)

 end

end

LEAST(E):从活结点表中选取成本估计函数 $\hat{c}(X)=f(h(X))+\hat{g}(X)$ 最小的结点赋值给E，并从表中删除该结点

- $f(h(X))=0$
- 若Y是X的子结点, $\hat{g}(Y) \leq \hat{g}(X)$
 - $\hat{c}(Y)=f(h(Y))+\hat{g}(Y) \leq \hat{c}(X)$
- 按照LC检索, 每次选取成本估计函数 $\hat{c}$ 的值最小的活结点作为下一个E-结点, 如果某些结点估计函数值相同, 选取最晚生成的活结点

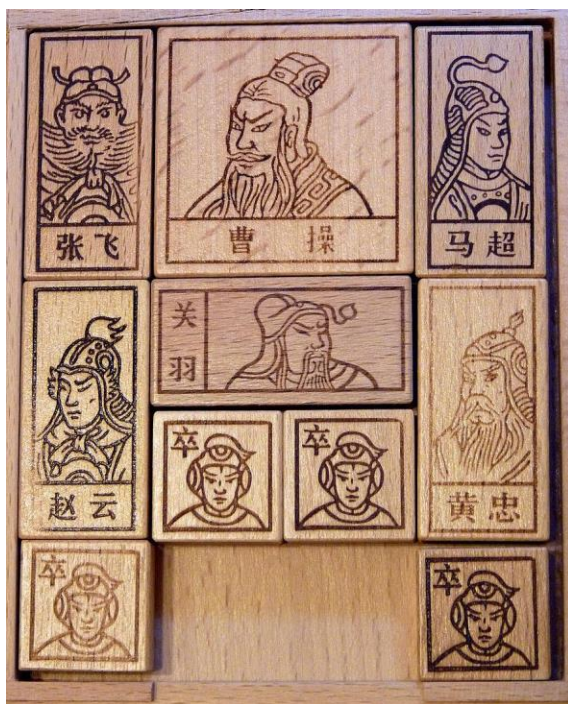
LIFO分支限界法

- 若 $f(h(X))$ =结点X的级数
- $\hat{g}(X)=0$
- $\hat{c}(X)=f(h(X))+\hat{g}(X)$ =结点X的级数
- 按照LC检索，每次选取成本估计函数 $\hat{c}$ 的值最小的活结点作为下一个E-结点，如果某些结点估计函数值相同，选取最早生成的活结点

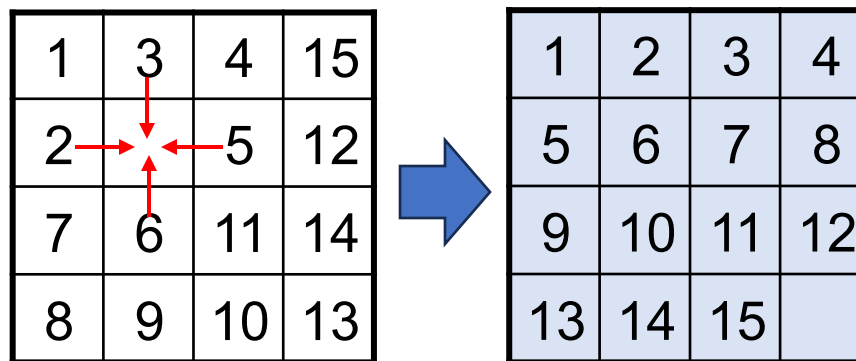
FIFO分支限界法

- 8.1 分支-限界法之检索方法
- 8.2 LC-检索
- 8.3 15-谜问题
- 8.4 再论LC-检索
- 8.5 0/1背包问题
- 8.6 分支-限界法之限界方法
- 8.7 小结

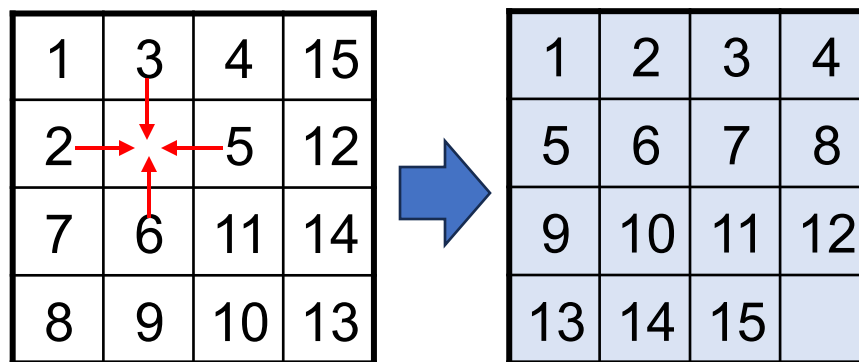
- 华容道：通过移动各个棋子，帮助曹操从初始位置移到棋盘最下方中部，从出口逃走



- 华容道：通过移动各个棋子，帮助曹操从初始位置移到棋盘最下方中部，从出口逃走
- 数字华容道（15-迷问题）：在一个分成16格的方形棋盘上放有15块编了号码的牌，要求通过一系列合法的移动转换成目标排列

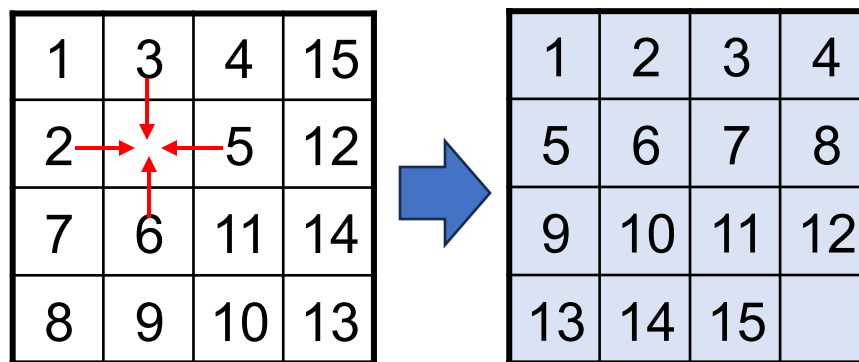


- 华容道：通过移动各个棋子，帮助曹操从初始位置移到棋盘最下方中部，从出口逃走
- 数字华容道（15-迷问题）：在一个分成16格的方形棋盘上放有15块编了号码的牌，要求通过一系列合法的移动转换成目标排列
- 输入：POSITION[1..16]，POSITION[i]是棋牌i的位置号，POSITION(16)表示空格的位置
- 输出：一系列合法的移动，使得POSITION = [1,2,...,16]



由于移动牌与移动空格等效，因此每次移动可以表示为空格的一次合法移动，按上、右、下、左的顺序进行

- 华容道：通过移动各个棋子，帮助曹操从初始位置移到棋盘最下方中部，从出口逃走
- 数字华容道（15-迷问题）：在一个分成16格的方形棋盘上放有15块编了号码的牌，要求通过一系列合法的移动转换成目标排列
- 输入：POSITION[1..16]，POSITION[i]是棋牌i的位置号，POSITION(16)表示空格的位置
- 输出：一系列合法的**空格**移动，使得POSITION = [1,2,...,16]



由于移动牌与移动空格等效，因此每次移动可以表示为空格的一次合法移动，按上、右、下、左的顺序进行

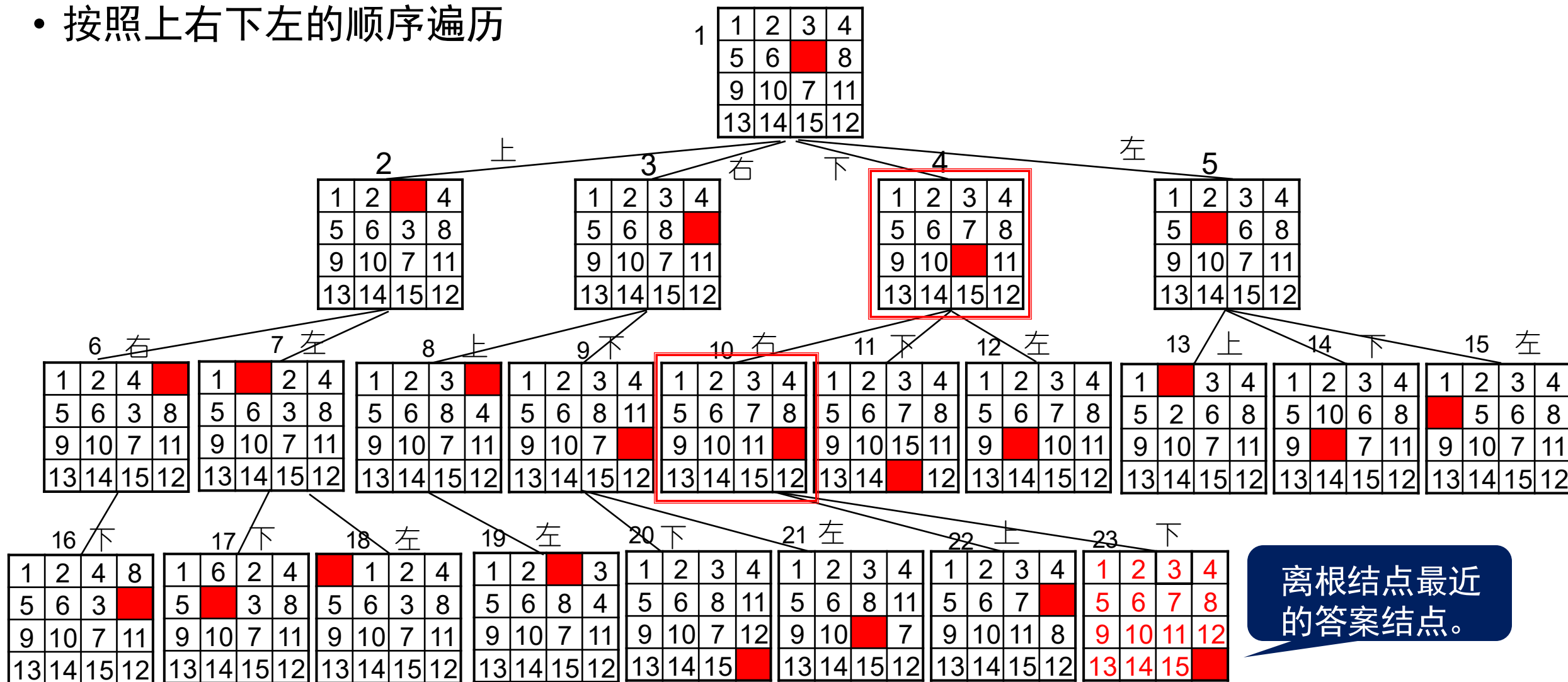
- 确定元组表达形式
 - 可变长元组: $(x_1, x_2, \dots, x_k)$
- 显式约束: $x_i \in \{\text{上}, \text{下}, \text{左}, \text{右}\}$
- 隐式约束: 基于 $x_1, x_2, \dots, x_k$ 对空格进行移动后, 使得 $\text{POSITION} = [1, 2, \dots, 16]$
- 状态空间树:
 - 问题状态结点 = 解状态结点: 所有通过移动空格能从初始排列得到的 POSITION
 - 答案状态结点:

不考虑限界函数

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

思考: 所有初始排列都能达到答案状态结点吗?

- 按照上右下左的顺序遍历

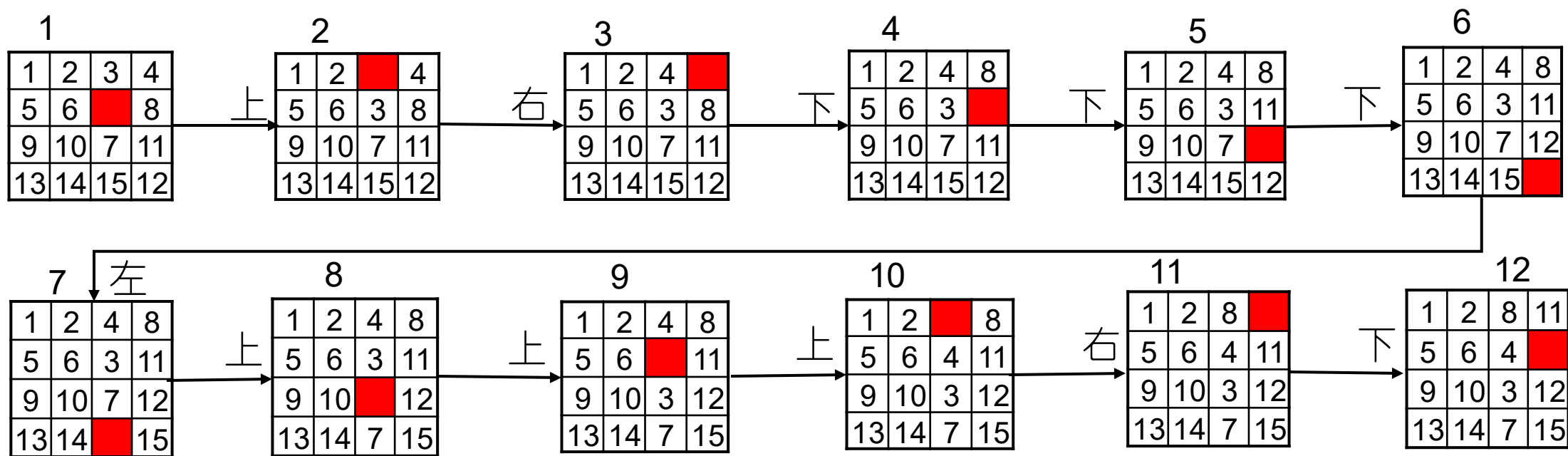


离根结点最近
的答案结点。

能找到离根最近的答案结点，但不管开局如何（不关心问题的具体实例），总是按千篇一律的顺序移动



- 按照上右下左的顺序遍历



不管开局如何（不关心问题的具体实例），总是采取由根开始的最左路径，呆板而盲目，搜索过程中有可能远离目标



- 定义 $c(X)$:
 - 将根结点到最近目标结点路径上的每个结点的成本设置为这条路径的长度
 - 其余结点成本赋值为 ∞
- LC-分支限界:
 - 首先将根结点作为E-结点，将成本为 ∞ 的子结点统统杀掉，只保留与根结点成本相同的子结点，并使其成为下一个E-结点
- 定义 $\hat{c}(X)$: $\hat{c}(X)=f(X)+\hat{g}(X)$
 - $f(X)$ 是由根到结点X路径的长度;
 - $\hat{g}(X)$ 是以X为根的子树中由X到目标状态的一条最短路径长度的估计值

找到离根结点最近的答案结点。

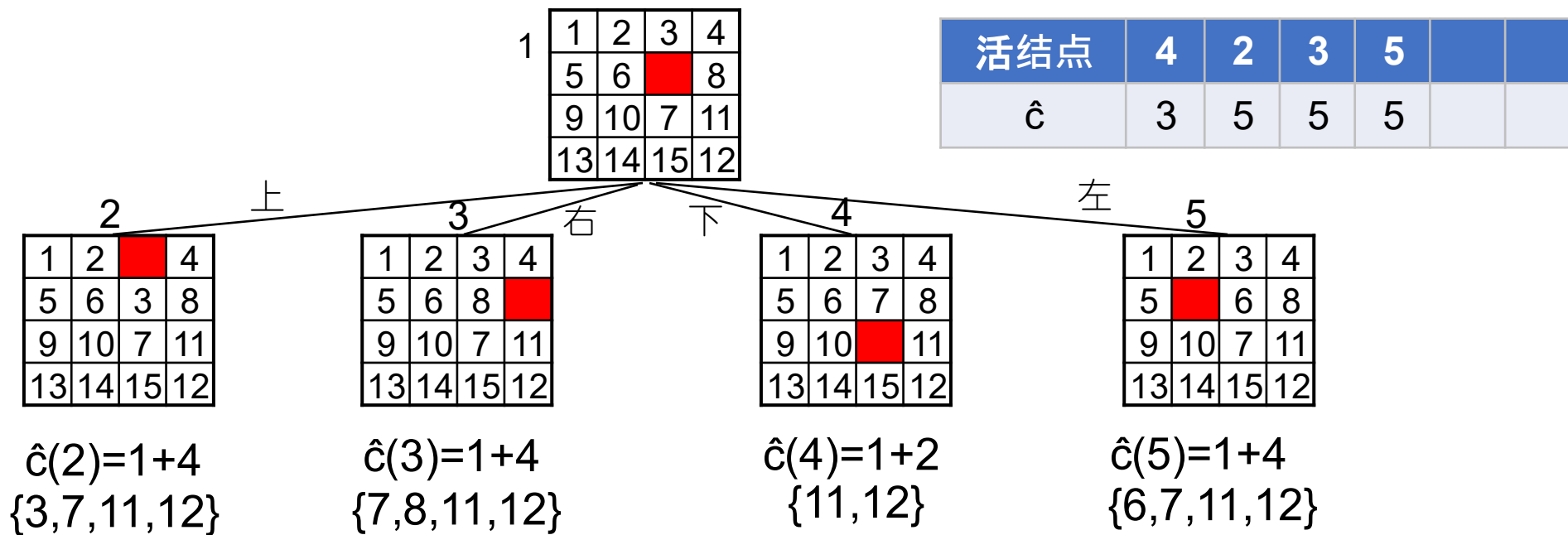
最小移动数

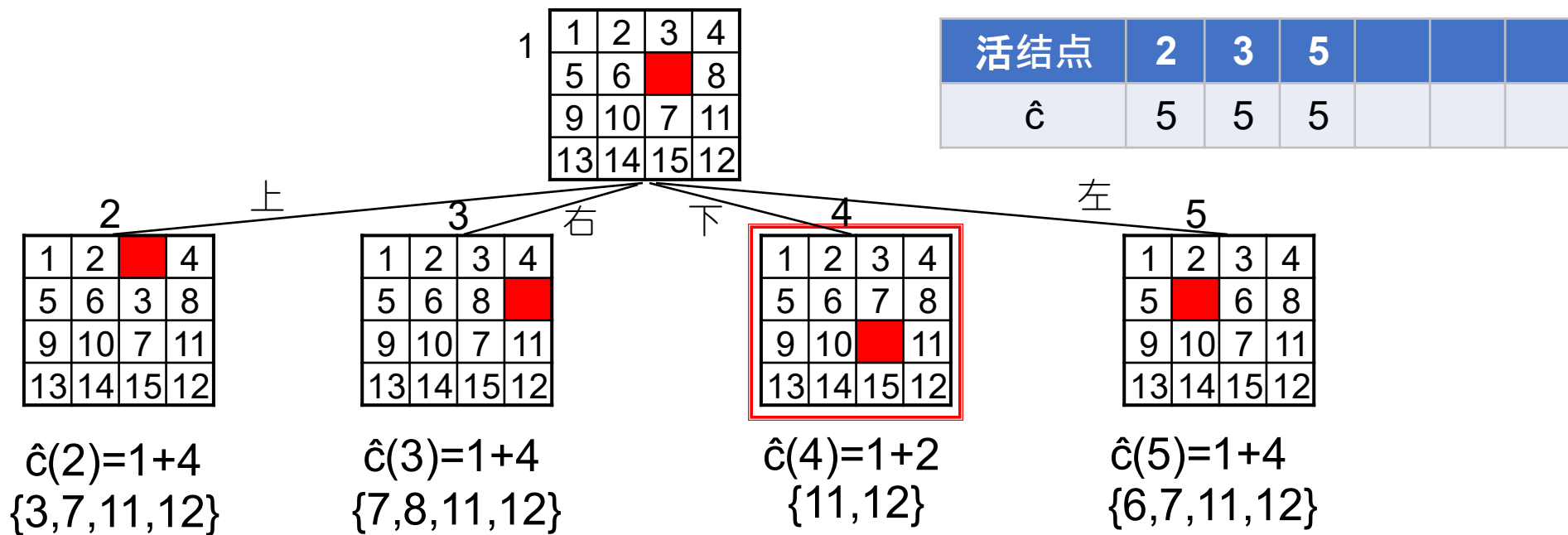
思考：如何定义 $\hat{g}(X)$ ？

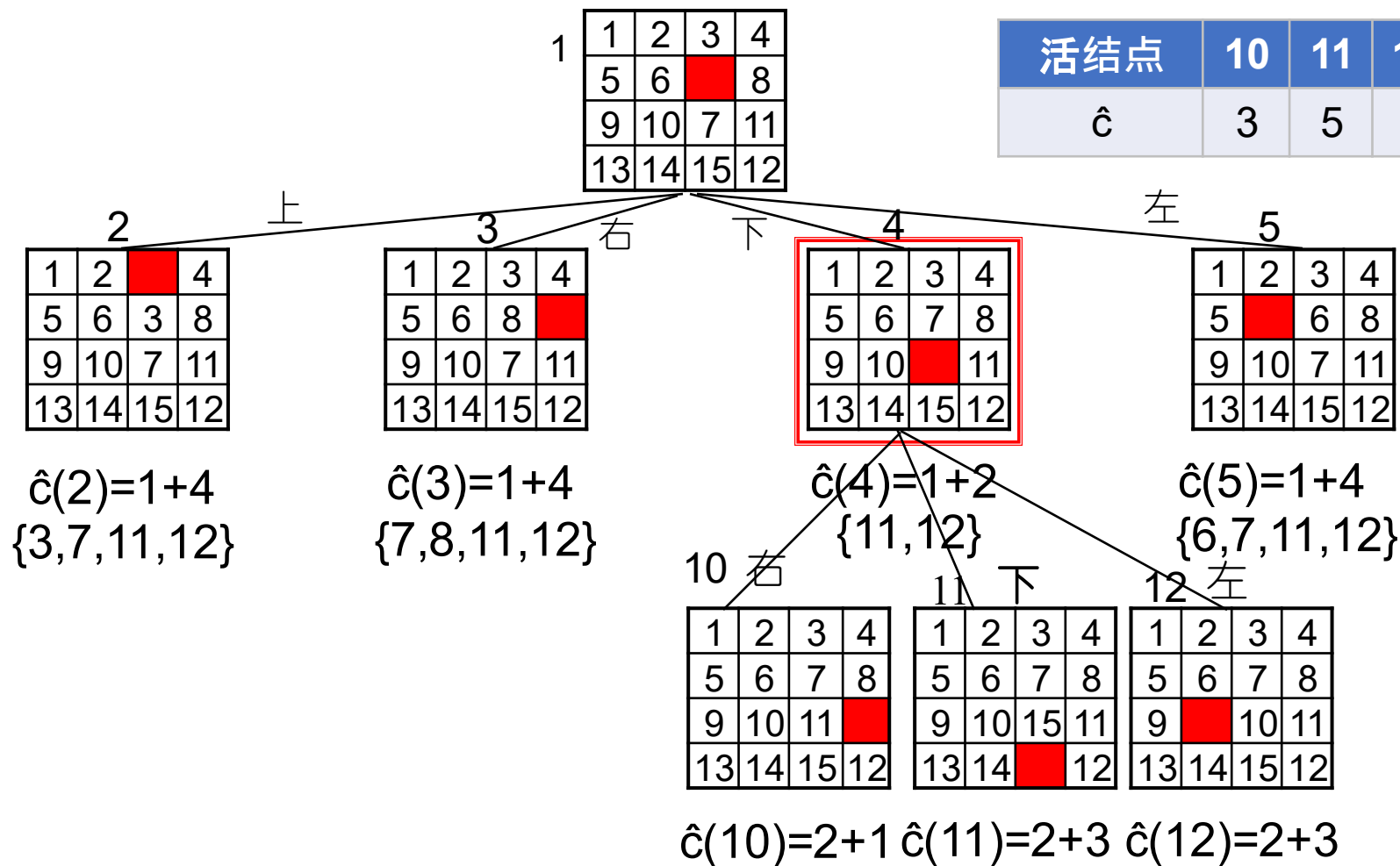
$\hat{g}(X)$ =不在其目标位置的非空白棋牌数目

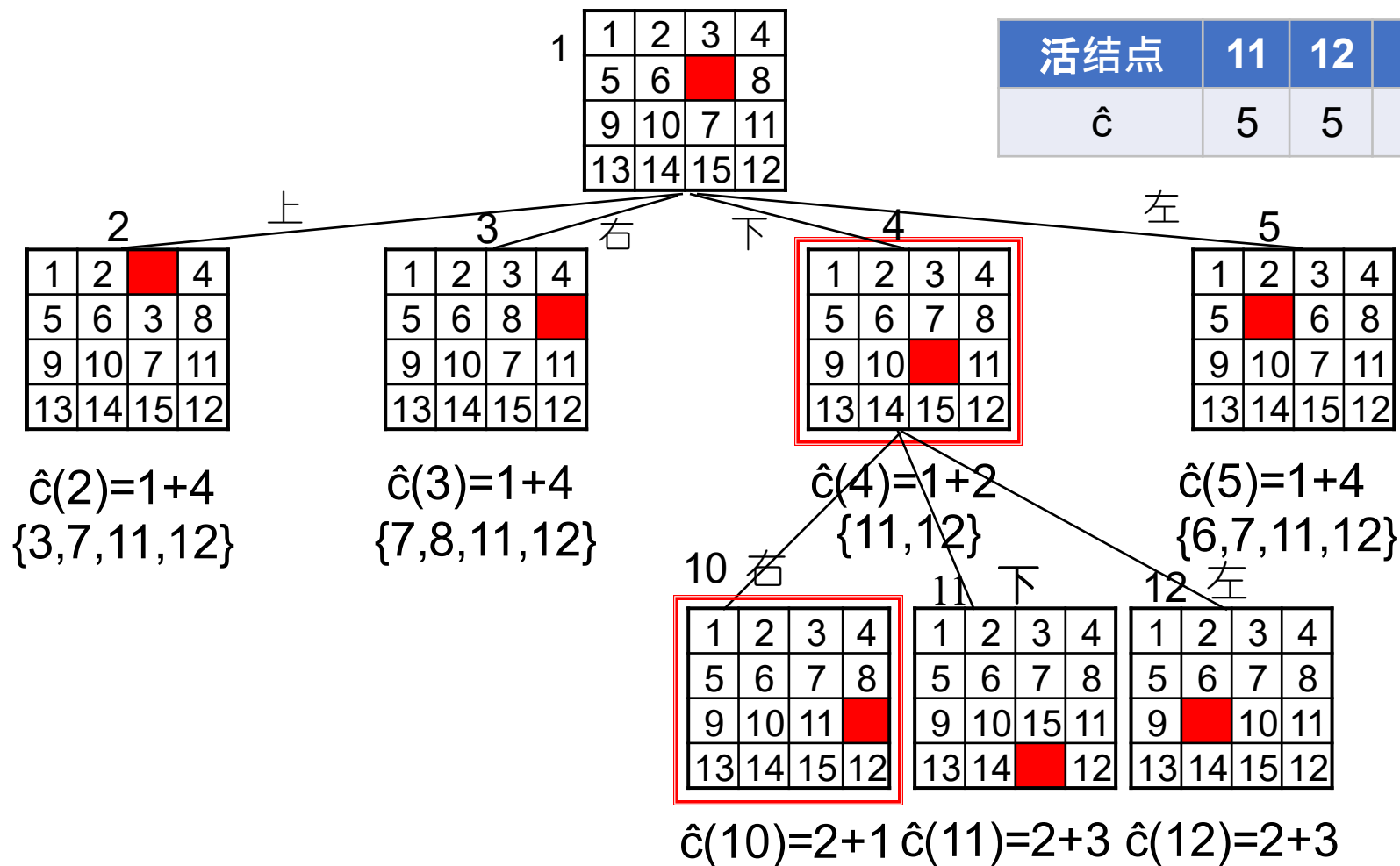
1	2	3	4
5	6		8
9	10	11	12
13	14	15	7

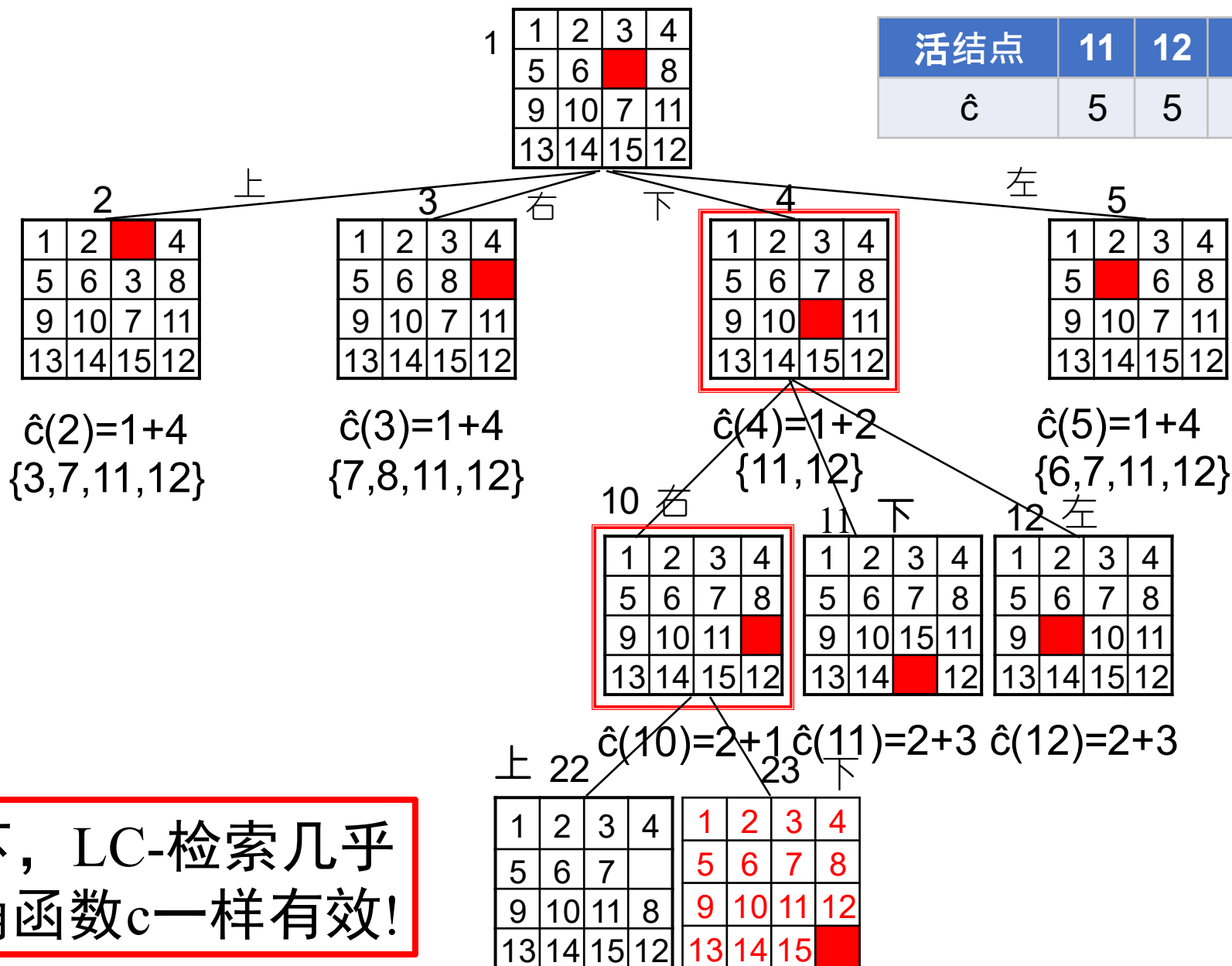
$\hat{g}(X)=1$











当前实例下，LC-检索几乎
和使用精确函数 c 一样有效!

- 8.1 分支-限界法之检索方法
- 8.2 LC-检索
- 8.3 15-谜问题
- 8.4 再论LC-检索
- 8.5 0/1背包问题
- 8.6 分支-限界法之限界方法
- 8.7 小结

- 成本估计函数 $\hat{c}$ 应该怎么设计？
 - 易于计算
 - 还有其他要求吗？

思考1：如何保证LC-检索一定能找到答案结点？

思考2：如何保证LC-检索一定能找到具有**最小成本**的答案结点？

- 成本估计函数 $\hat{c}$ 应该怎么设计？
 - 易于计算
- 如何保证LC-检索一定能找到具有最小成本的答案结点？
 - 对于答案结点 X ，有 $\hat{c}(X) = c(X)$
 - $\hat{c}$ 只影响LEAST(E)的结果：不关心 $\hat{c}$ 的具体值，而是结点 $\hat{c}$ 值的大小关系
 - $c(X) < c(Y)$ 时，有 $\hat{c}(X) < \hat{c}(Y)$

LEAST(E):从活结点表中选取成本估计函数 $\hat{c}(X)=f(h(X))+ \hat{g}(X)$ **最小**的结点赋值给 E ，并从表中删除该结点

条件 “ $c(X) < c(Y)$ 时，有 $\hat{c}(X) < \hat{c}(Y)$ ” 能保证LC-检索一定能找到具有最小成本的答案结点！

- 成本估计函数 $\hat{c}$ 应该怎么设计？
 - 易于计算
- 如何保证LC-检索一定能找到具有最小成本的答案结点？
 - 对于答案结点X，有 $\hat{c}(X) = c(X)$
 - $\hat{c}$ 只影响LEAST(E)的结果：不关心 $\hat{c}$ 的具体值，而是结点 $\hat{c}$ 值的大小关系
 - $c(X) < c(Y)$ 时，有 $\hat{c}(X) < \hat{c}(Y)$  与原问题具有类似的复杂度

- 成本估计函数 $\hat{c}$ 应该怎么设计？
 - 易于计算
- 如何保证LC-检索一定能找到具有最小成本的答案结点？
 - 对于答案结点 X ，有 $\hat{c}(X) = c(X)$
 - $\hat{c}$ 只影响LEAST(E)的结果：不关心 $\hat{c}$ 的具体值，而是结点 $\hat{c}$ 值的大小关系
 - ~~• $c(X) < c(Y)$ 时，有 $\hat{c}(X) < \hat{c}(Y)$~~
 - 对于每一个结点 X ， $\hat{c}(X) \leq c(X)$

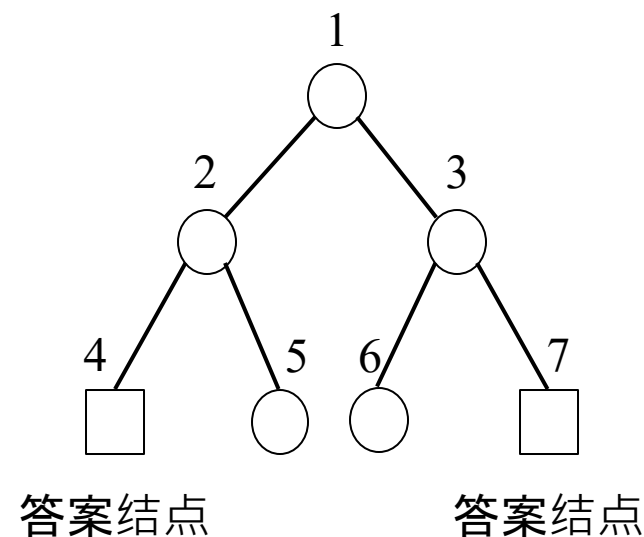
与原问题具有类似的复杂度

成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

```
...
loop
  for E的每个儿子X do
    if X是答案结点 then 输出从X到T的那条路径; return end
    if B(X) then ADD(X); PARENT(X)←E end
  end
  if 不再有活结点 then print("no answer node"); return end
  LEAST(E)
end
...
```



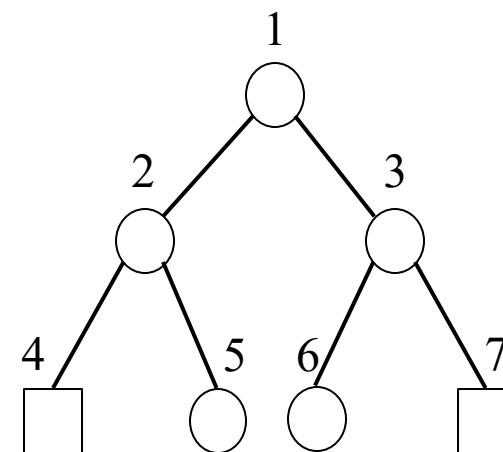
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	1						
$\hat{c}$	0						

```
...
loop
  for E的每个儿子X do
    if X是答案结点 then 输出从X到T的那条路径; return end
    if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
  end
  if 不再有活结点 then print("no answer node"); return end
  LEAST(E)
end
...
```



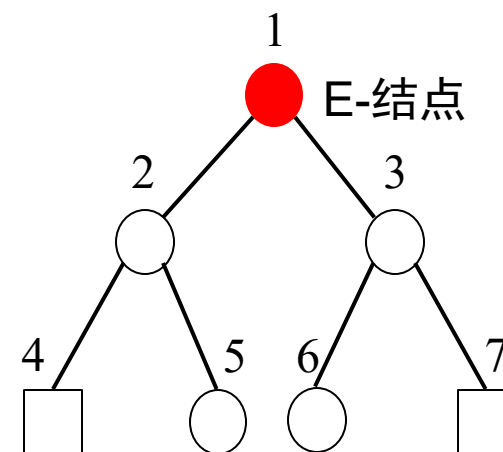
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点							
$\hat{c}$							

```
...
loop
  for E的每个儿子X do
    if X是答案结点 then 输出从X到T的那条路径; return end
    if B(X) then ADD(X); PARENT(X)←E end
  end
  if 不再有活结点 then print("no answer node"); return end
  LEAST(E)
end
...
```



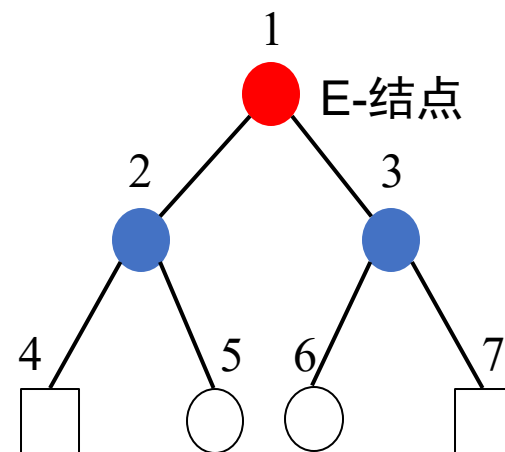
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	2	3					
$\hat{c}$	2	4					

...
loop
 for E的每个儿子X do
 if X是答案结点 then 输出从X到T的那条路径; return end
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



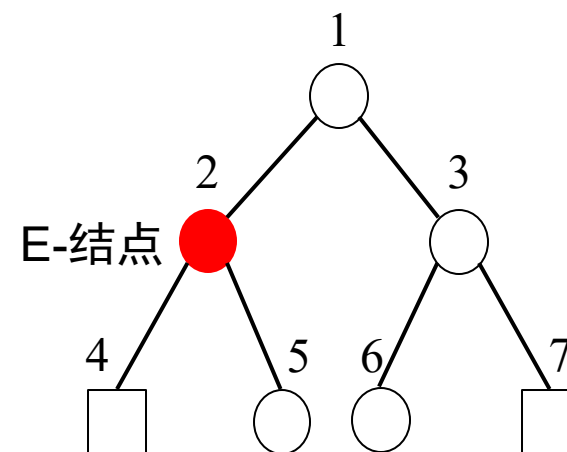
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	3						
$\hat{c}$	4						

...
loop
 for E的每个儿子X do
 if X是答案结点 then 输出从X到T的那条路径; return end
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



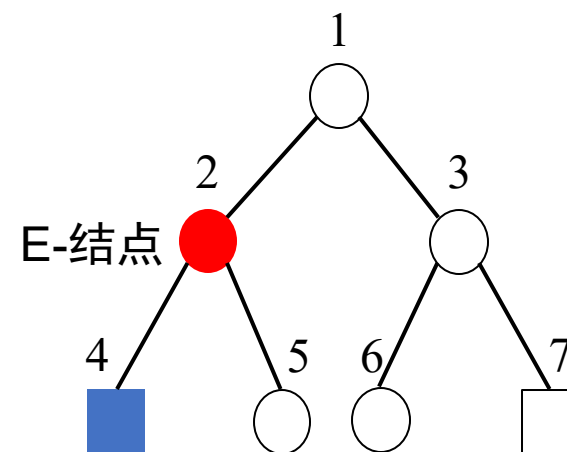
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计？
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	3						
$\hat{c}$	4						

```
...
loop
  for E的每个儿子X do
    if X是答案结点 then 输出从X到T的那条路径; return end
    if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
  end
  if 不再有活结点 then print("no answer node"); return end
  LEAST(E)
end
...
```



找到答案结点,
但并非成本最小!

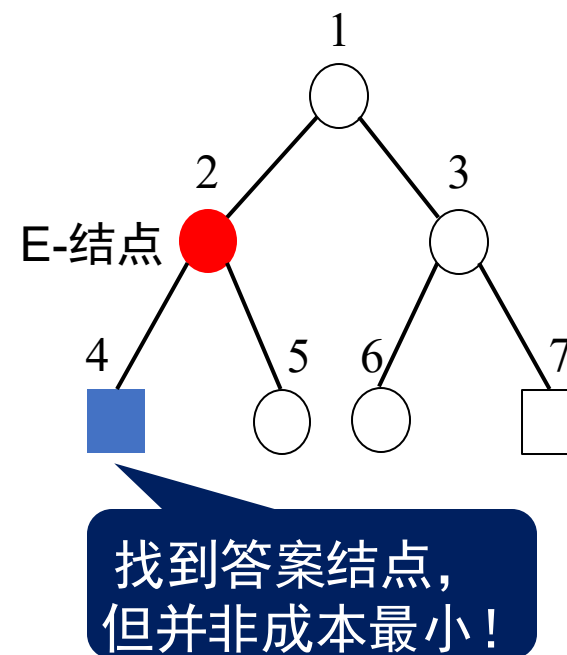
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	3						
$\hat{c}$	4						

```
...
loop
  if E是答案结点 then 输出从E到T的那条路径; return end
  for E的每个儿子X do
    if X是答案结点 then 输出从X到T的那条路径; return end
    if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
  end
  if 不再有活结点 then print("no answer node"); return end
  LEAST(E)
end
...
```



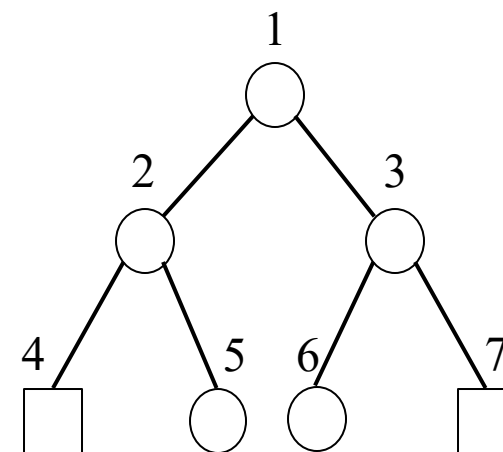
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

...

```
loop
  if E是答案结点 then 输出从E到T的那条路径; return end
  for E的每个儿子X do
    if B(X) then ADD(X); PARENT(X)←E end
  end
  if 不再有活结点 then print("no answer node"); return end
  LEAST(E)
end
...
```



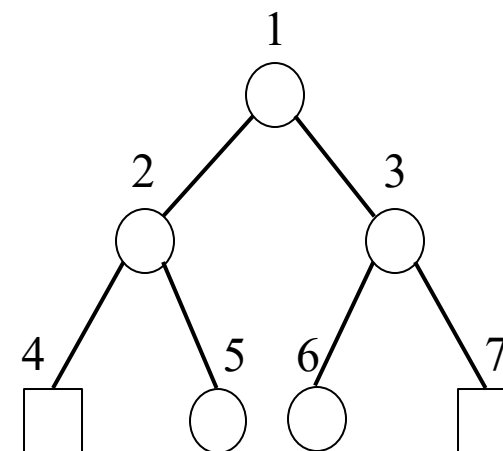
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	1						
$\hat{c}$	0						

...
loop
 if E是答案结点 then 输出从E到T的那条路径; return end
 for E的每个儿子X do
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



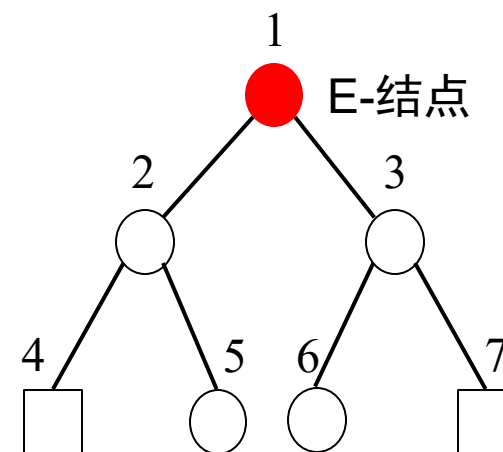
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点							
$\hat{c}$							

...
loop
 if E是答案结点 then 输出从E到T的那条路径; return end
 for E的每个儿子X do
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



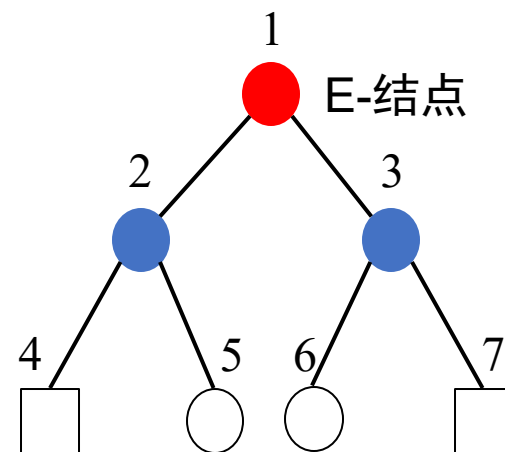
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	2	3					
$\hat{c}$	2	4					

...
loop
 if E是答案结点 then 输出从E到T的那条路径; return end
 for E的每个儿子X do
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



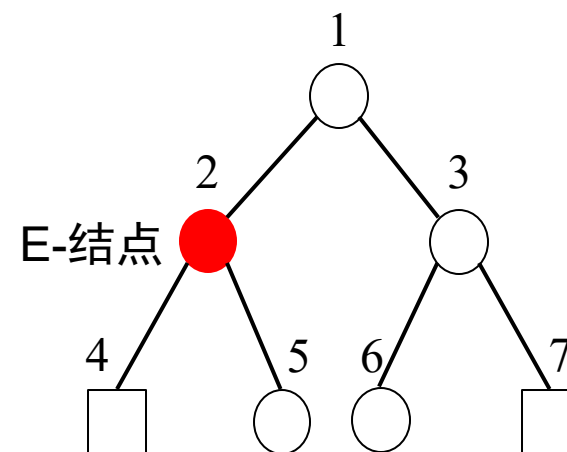
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	3						
$\hat{c}$	4						

...
loop
 if E是答案结点 then 输出从E到T的那条路径; return end
 for E的每个儿子X do
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



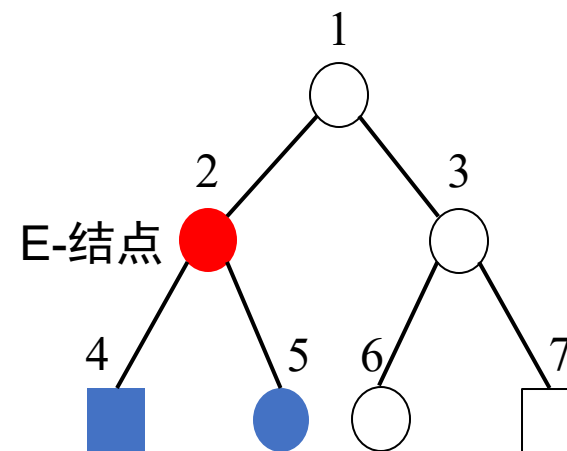
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	3	4	5				
$\hat{c}$	4	20	∞				

...
loop
 if E是答案结点 then 输出从E到T的那条路径; return end
 for E的每个儿子X do
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



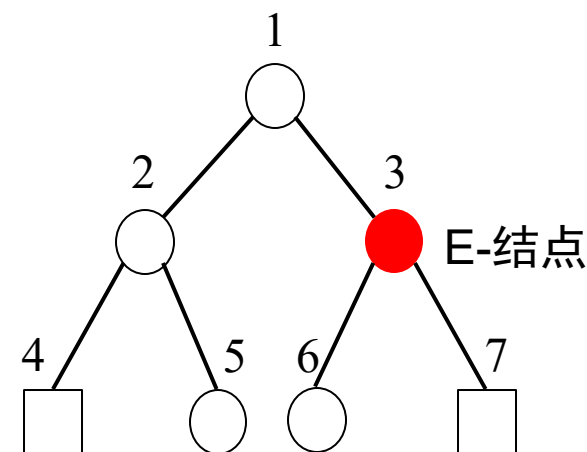
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	4	5					
$\hat{c}$	20	∞					

...
loop
 if E是答案结点 then 输出从E到T的那条路径; return end
 for E的每个儿子X do
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



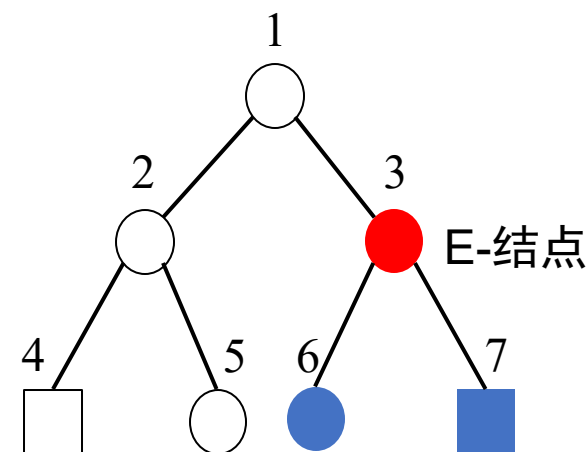
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	7	4	5	6			
$\hat{c}$	10	20	∞	∞			

...
loop
 if E是答案结点 then 输出从E到T的那条路径; return end
 for E的每个儿子X do
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



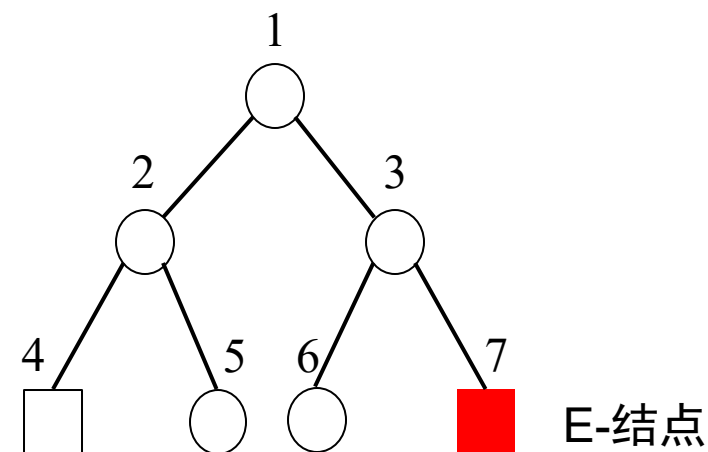
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	4	5	6				
$\hat{c}$	20	∞	∞				

...
loop
 if E是答案结点 then 输出从E到T的那条路径; return end
 for E的每个儿子X do
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



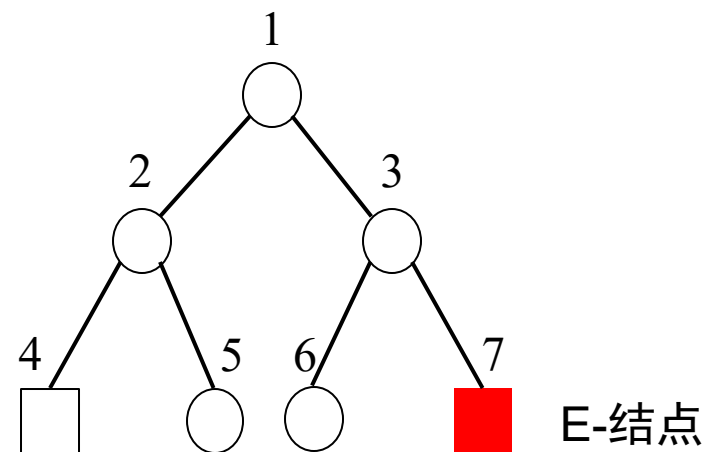
成本估计函数 $\hat{c}$ 的特性

- 成本估计函数 $\hat{c}$ 应该怎么设计?
 - 易于计算
 - 对于答案结点X, 有 $\hat{c}(X) = c(X)$
 - 对于每一个结点X, $\hat{c}(X) \leq c(X)$

结点	1	2	3	4	5	6	7
c	10	20	10	20	∞	∞	10
$\hat{c}$	0	2	4	20	∞	∞	10

活结点	4	5	6				
$\hat{c}$	20	∞	∞				

...
loop
 if E是答案结点 then 输出从E到T的那条路径; return end
 for E的每个儿子X do
 if B(X) then ADD(X); PARENT(X) $\leftarrow$ E end
 end
 if 不再有活结点 then print("no answer node"); return end
 LEAST(E)
end
...



找到成本最小的
答案结点!

求最小成本的LC-检索算法

```
procedure LC(T,  $\hat{c}$ ) //为找出最小成本答案结点
  E  $\leftarrow$  T, 将活结点表初始化为空
  loop
    if E是答案结点 then 输出从E到T的那条路径; return; end
    for E的每个儿子X do
      if B(X) then call ADD(X); PARENT(X)  $\leftarrow$  E; end
    end
    if 不再有活结点 then print("no answer node"); return; end
    LEAST(E)
  end
end
```

选择 $\hat{c}$ 最小的

- 对于活结点表中的每一个结点L，一定有 $\hat{c}(E) \leq \hat{c}(L)$ 。由 $\hat{c}$ 定义知，E是答案结点时 $c(E) = \hat{c}(E)$ ，则 $c(E) = \hat{c}(E) \leq \hat{c}(L) \leq c(L)$
- 因此，算法LC一定能检索到成本最小的答案结点

- 在满足约束条件的解中找出使得某一**目标函数**达到极大或极小的解，即某种意义下的最优解
- 若优化问题是极大化问题，改变目标函数，将问题转化为**极小化**问题
- 以目标函数作为**成本函数** c
- 问题转化为寻找状态空间树中**最小成本**的答案结点

此时， $\hat{c}(X)$ 不是估计到达一个答案结点的难易程度，
而是估计目标函数！

- 如果X是答案结点
 - ~~$c(X)$ =状态空间树的根结点到X的成本(即所用的代价, 可以是级数、计算复杂度等)~~
 - 若X是叶子结点, $c(X)$ =X的目标函数值
 - 否则, $c(X)$ =子树X中具有最小成本的答案结点的成本
- 如果X不是答案结点且子树X不包含任何答案结点
 - 则 $c(X)=\infty$
- 如果X不是答案结点, 但子树X中包含答案结点
 - $c(X)$ =子树X中具有最小成本的答案结点的成本

常见于可变k-元组状态空间树, X是部分解, 子孙中有可能有成本更小的结点

- 8.1 分支-限界法之检索方法
- 8.2 LC-检索
- 8.3 15-谜问题
- 8.4 再论LC-检索
- **8.5 0/1背包问题**
- 8.6 分支-限界法之限界方法
- 8.7 小结

- 0/1背包问题要求物品或者整件装入背包中, 或者根本不装入(即不能装入物品的一部分), 所以 x_i 限定只能取0或1值
- 输入: 物品个数 n , 各物品价值和体积 p_i, w_i , 背包容量 M
- 输出: X_i 表示第 i 件物品是否装入背包, 使得物品效益和最大

$$\begin{array}{ccc} \max_{1 \leq i \leq n} p_i X_i, & s.t., & \min_{1 \leq i \leq n} -p_i X_i, & s.t., \\ \sum_{1 \leq i \leq n} w_i X_i \leq M, X_i = 0,1 & \longrightarrow & \sum_{1 \leq i \leq n} w_i X_i \leq M, X_i = 0,1 \end{array}$$

- 解空间的表示方法

- n -元组($X_1, \dots, X_n$)

- 显式约束条件: $X_i = 0/1, 1 \leq i \leq n$

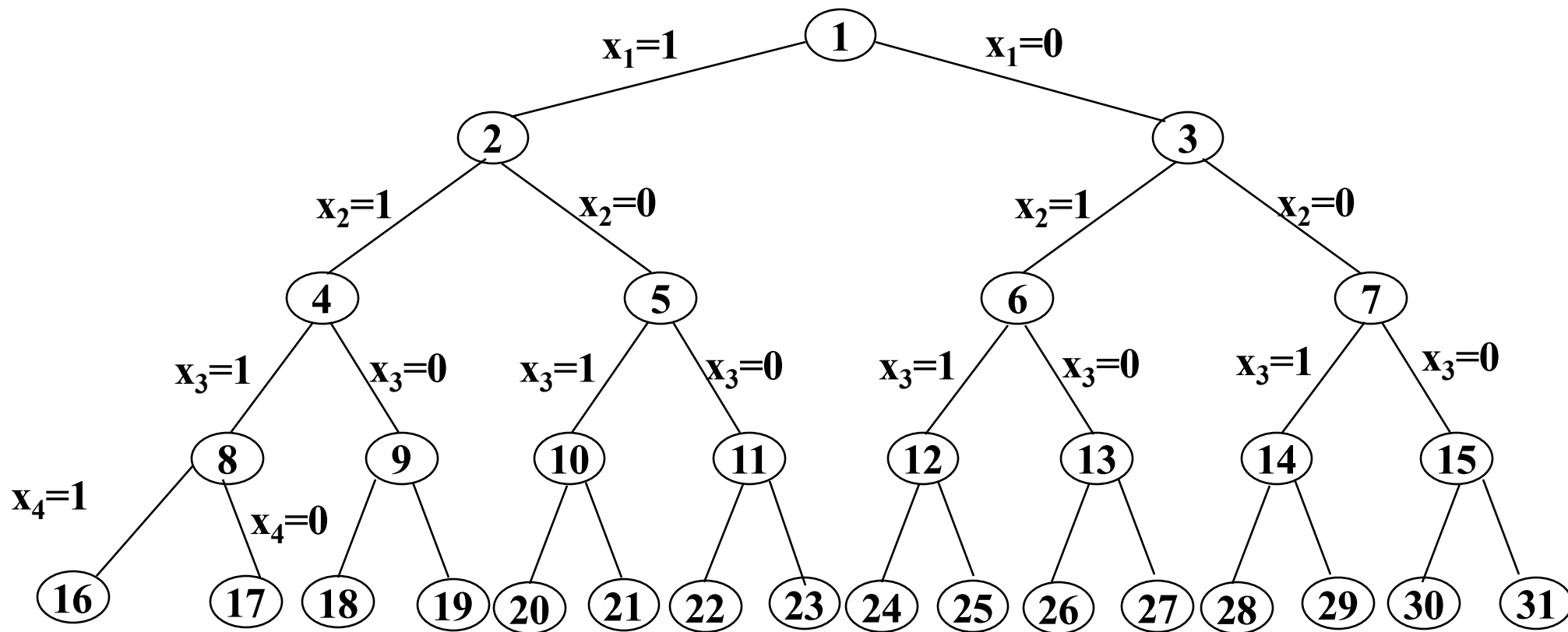
- 隐式约束条件: 选中的物品不违反 M 的限制

限界函数 B

- 目标函数: 选中的物品效益和极大化

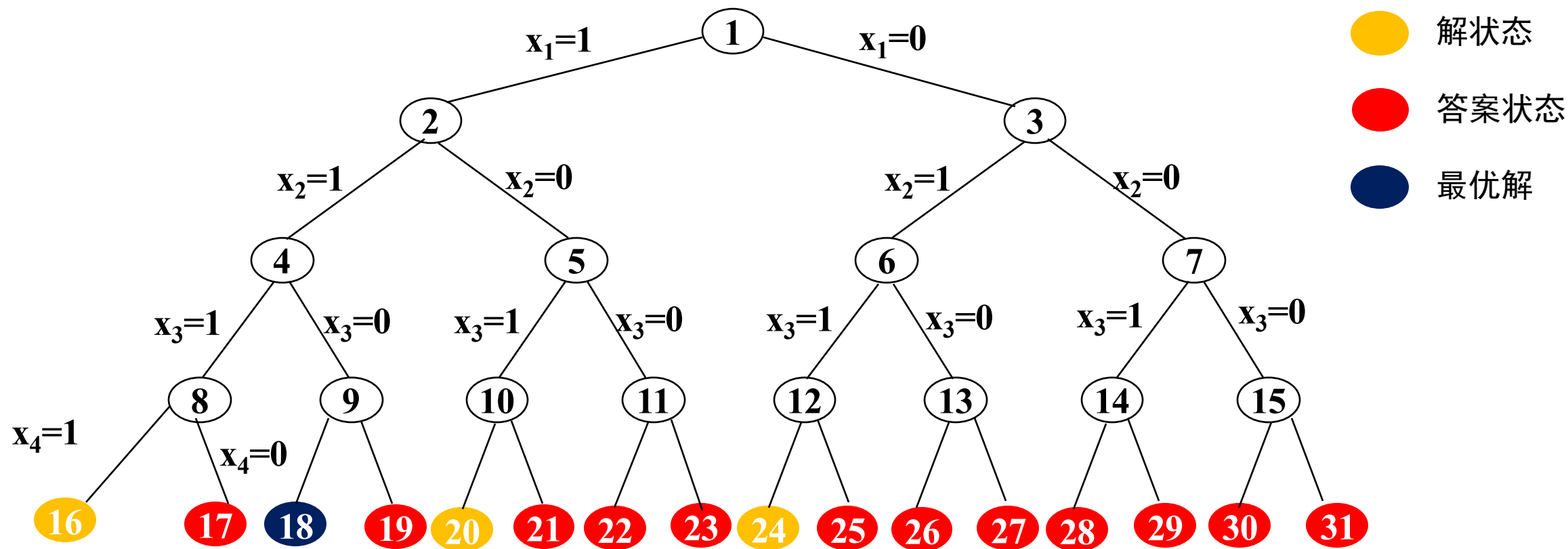
成本函数 c

- $n=4$, $M=15$, $(p_1, p_2, p_3, p_4)=(10, 10, 12, 18)$, $(w_1, w_2, w_3, w_4)=(2, 4, 6, 9)$



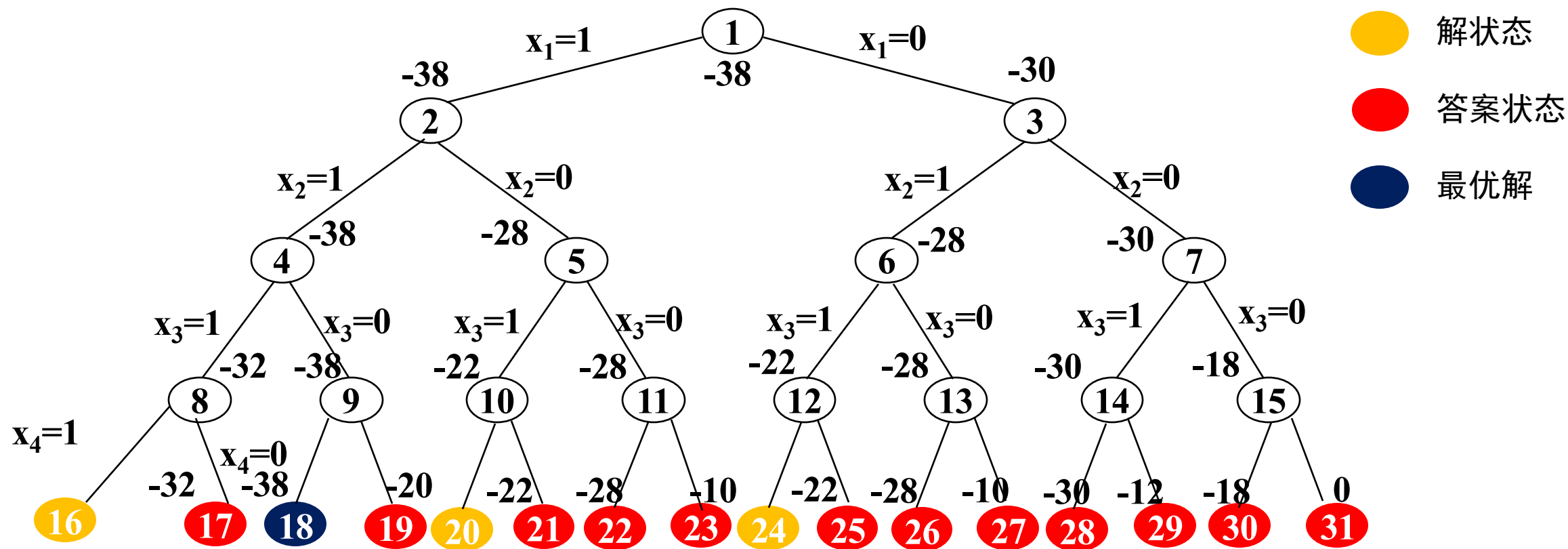
- 问题的解空间由 2^n 个不同的 n 元组向量组成

- $n=4$, $M=15$, $(p_1, p_2, p_3, p_4)=(10, 10, 12, 18)$, $(w_1, w_2, w_3, w_4)=(2, 4, 6, 9)$



- 问题的解空间由 2^n 个不同的 n 元组向量组成
- 满足 $\leq M$ 的叶节点是答案节点，最大价值的答案节点是问题的最优解

- $n=4$, $M=15$, $(p_1, p_2, p_3, p_4)=(10, 10, 12, 18)$, $(w_1, w_2, w_3, w_4)=(2, 4, 6, 9)$



- 问题的解空间由 2^n 个不同的 n 元组向量组成
- 满足 $\leq M$ 的叶节点是答案节点，最大价值的答案节点是问题的最优解

成本估计函数 $\hat{c}$ 需满足的性质

- 易于计算
- 对于答案结点 X , 有 $\hat{c}(X) = c(X)$
- 对于每一个结点 X , $\hat{c}(X) \leq c(X)$

- 结点 $\langle X_1, X_2, \dots, X_k \rangle$ 的成本估计函数为：固定 $\langle X_1, X_2, \dots, X_k \rangle$, 无论 $X_{k+1}, \dots, X_n$ 取何值, 所能获得的最大效益值的上界（最小目标函数值的下界）

$$\min_{k < i \leq n} -p_i X_i, \quad s.t.,$$
$$\sum_{k < i \leq n} w_i X_i \leq M - \sum_{1 \leq i \leq k} w_i X_i, X_i = 0, 1$$

最优值



$$\min_{k < i \leq n} -p_i X_i, \quad s.t.,$$
$$\sum_{k < i \leq n} w_i X_i \leq M - \sum_{1 \leq i \leq k} w_i X_i$$

最优值

背包问题
贪心法！

Procedure LB (p,w,k,M)

// 物品已按照单位价值进行非升序排列

// p: 前k个决策对应获得的价值; w: 前k个决策对应消耗的重量

// 基于贪心方法考虑物品k+1到n, 在M-w约束条件下的贪心解

pp $\leftarrow$ p; ww $\leftarrow$ w

for i $\leftarrow$ k+1 to n do

if ww+W(i) $\leq$ M then ww $\leftarrow$ ww+W(i); pp $\leftarrow$ pp+P(i)

else return -(pp+(M-ww)*P(i)/W(i)) end

end

return -pp

end

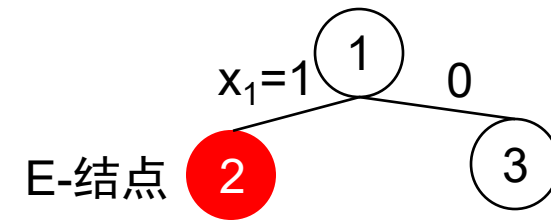
- $n=4, M=15, (p_1, p_2, p_3, p_4)=(10, 10, 12, 18), (w_1, w_2, w_3, w_4)=(2, 4, 6, 9)$

活结点	贪心策略	$\hat{c}$
1	1,1,1,1/3	-38

1 E-结点

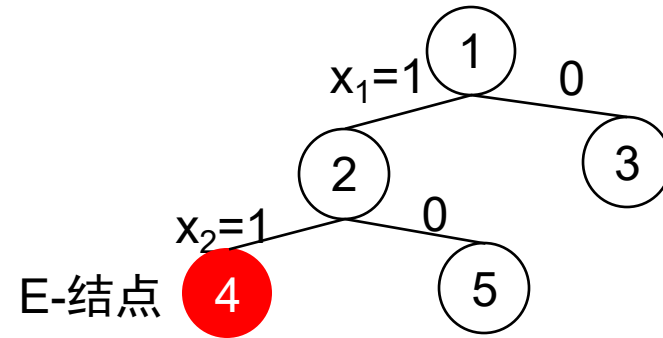
- $n=4, M=15, (p_1, p_2, p_3, p_4)=(10, 10, 12, 18), (w_1, w_2, w_3, w_4)=(2, 4, 6, 9)$

活结点	贪心策略	$\hat{c}$
2	1,1,1,1/3	-38
3	0,1,1,5/9	-32



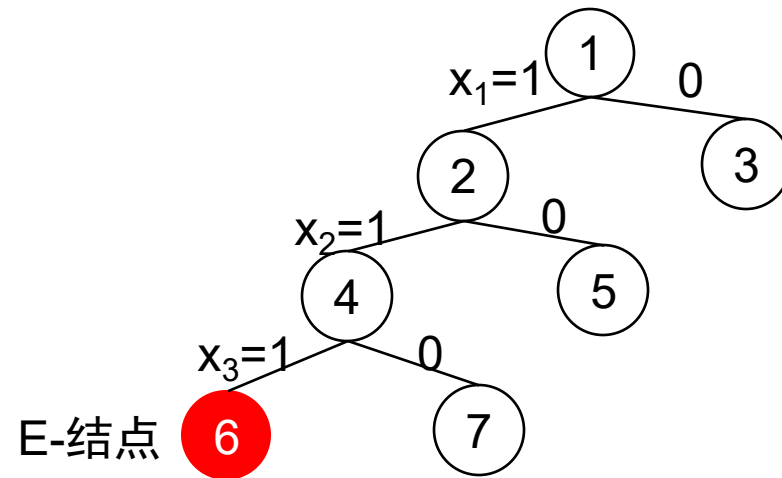
- $n=4$, $M=15$, $(p_1, p_2, p_3, p_4)=(10, 10, 12, 18)$, $(w_1, w_2, w_3, w_4)=(2, 4, 6, 9)$

活结点	贪心策略	$\hat{c}$
4	1,1,1,1/3	-38
5	1,0,1,7/9	-36
3	0,1,1,5/9	-32



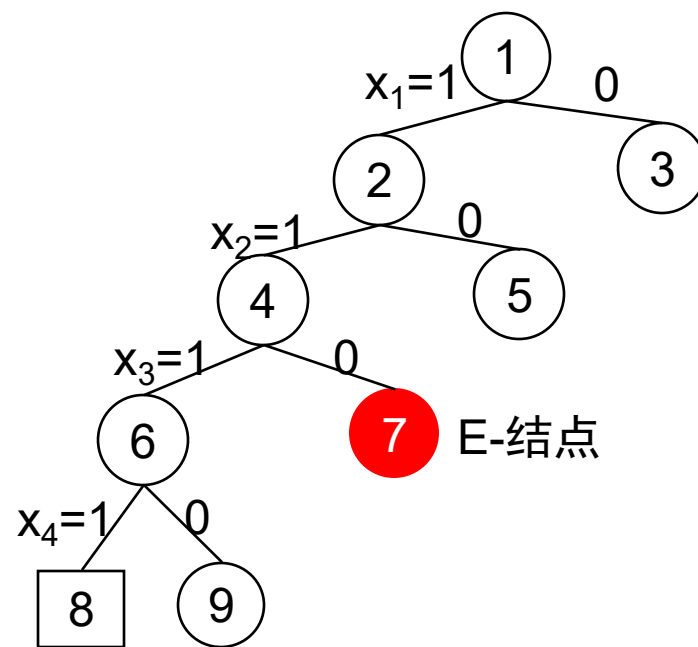
- $n=4, M=15, (p_1, p_2, p_3, p_4)=(10, 10, 12, 18), (w_1, w_2, w_3, w_4)=(2, 4, 6, 9)$

活结点	贪心策略	$\hat{c}$
6	1,1,1,1/3	-38
7	1,1,0,1	-38
5	1,0,1,7/9	-36
3	0,1,1,5/9	-32



- $n=4, M=15, (p_1, p_2, p_3, p_4)=(10, 10, 12, 18), (w_1, w_2, w_3, w_4)=(2, 4, 6, 9)$

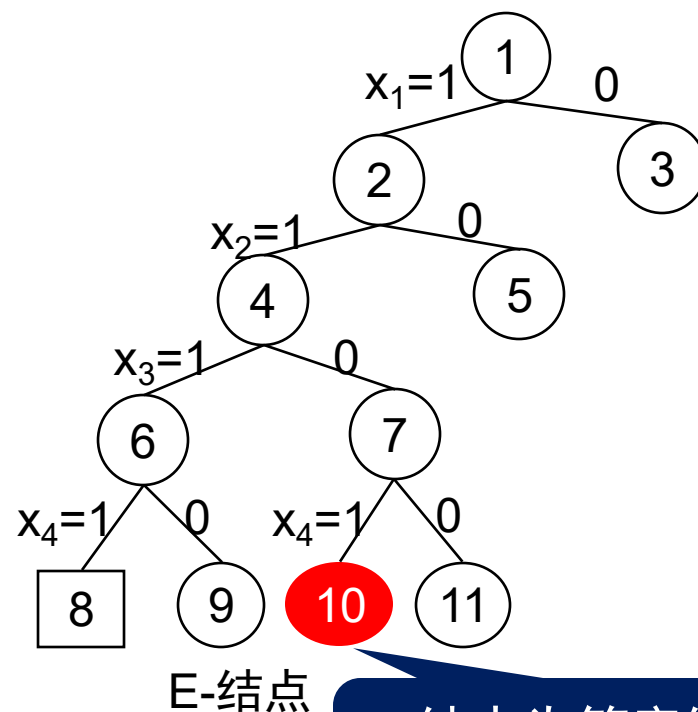
活结点	贪心策略	$\hat{c}$
7	1,1,0,1	-38
5	1,0,1,7/9	-36
3	0,1,1,5/9	-32
9	1,1,1,0	-32



不满足限界函数，kill

- $n=4, M=15, (p_1, p_2, p_3, p_4)=(10, 10, 12, 18), (w_1, w_2, w_3, w_4)=(2, 4, 6, 9)$

活结点	贪心策略	$\hat{c}$
10	1,1,0,1	-38
5	1,0,1,7/9	-36
3	0,1,1,5/9	-32
9	1,1,1,0	-32
11	1,1,0,0	-20



E结点为答案结点
成本最小

- 8.1 分支-限界法之检索方法
- 8.2 LC-检索
- 8.3 15-谜问题
- 8.4 再论LC-检索
- 8.5 0/1背包问题
- 8.6 分支-限界法之限界方法
- 8.7 小结

- 假设有 n 个作业和一台处理机，每个作业 i 由一个三元组 (p_i, d_i, t_i) 表示，表示作业需要 t_i 个时间处理完毕，如果在期限 d_i 之前没有完成则要交付 p_i 的**罚款**
- 问题目标：从这 n 个作业中选取一个子集合 J ，使 J 中作业都能在相应的期限内完成，而不在 J 中的作业罚款总数最小

$$\min_{S \subseteq \{1, 2, \dots, n\}} \sum_{i \notin S} p_i, \quad s.t.,$$

S 中的作业能够在各自期限内处理完毕

- 解空间的表示方法

- 大小可变的 k -元组 $(X_1, \dots, X_k)$, $k \leq n$

- 显式约束条件：元组 X_j 表示选中的作业下标, $X_i \leq X_{i+1}$, $1 \leq i < k$ 保证不重复

- 隐式约束条件：作业能在期限前完成

限界函数B

- 目标函数：未选中的作业罚款总数最小

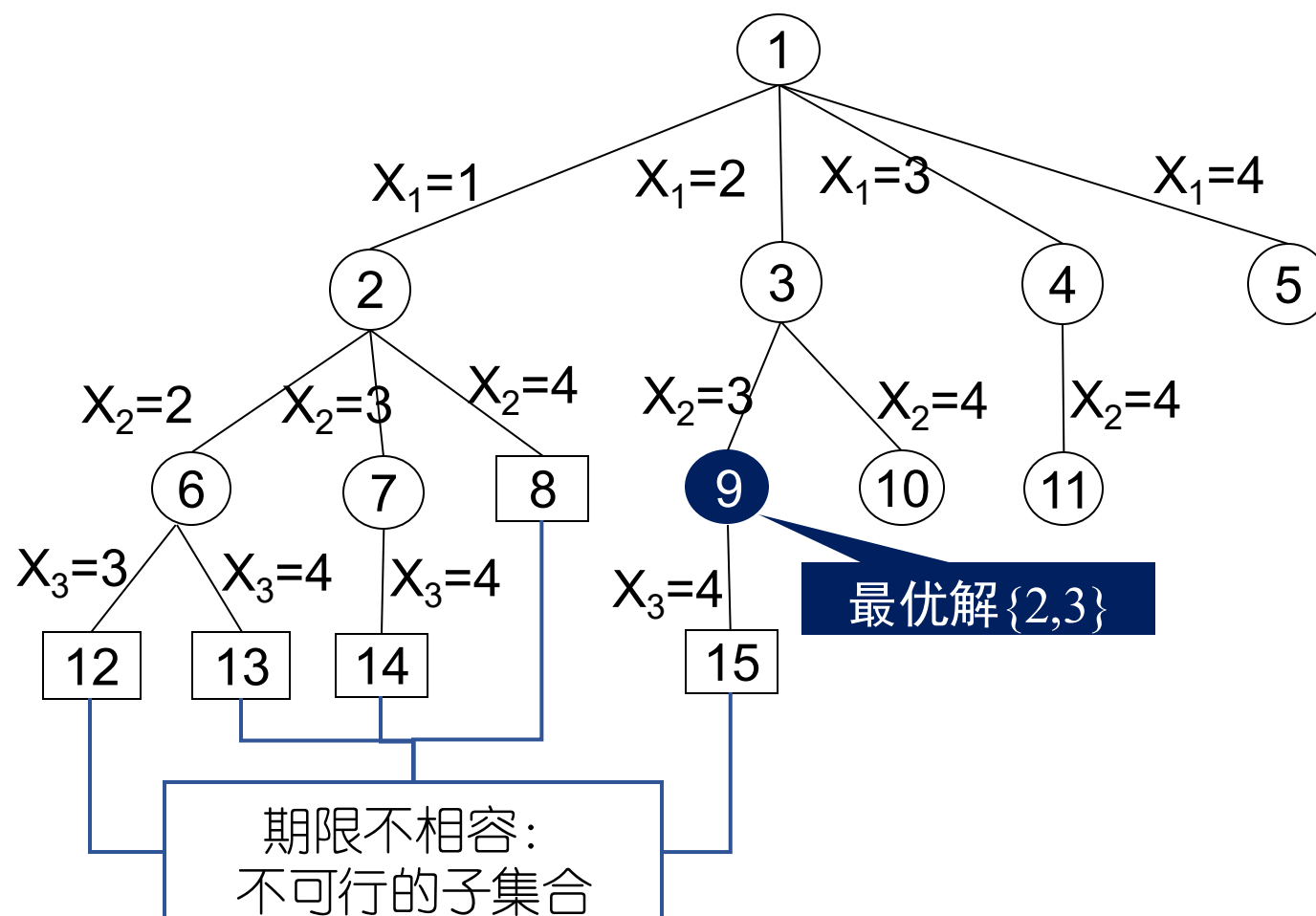
成本函数c

作业	1	2	3	4
p	5	10	6	3
d	1	3	2	1
t	1	2	1	1

- 状态空间树
 - 共计 2^n 个结点
 - 圆形结点表示答案结点(可行解)
 - 方形结点表示不可行结点

成本估计函数 $\hat{c}$ 需满足的性质

- 易于计算
- 对于答案结点 X , 有 $\hat{c}(X) = c(X)$
- 对于每一个结点 X , $\hat{c}(X) \leq c(X)$



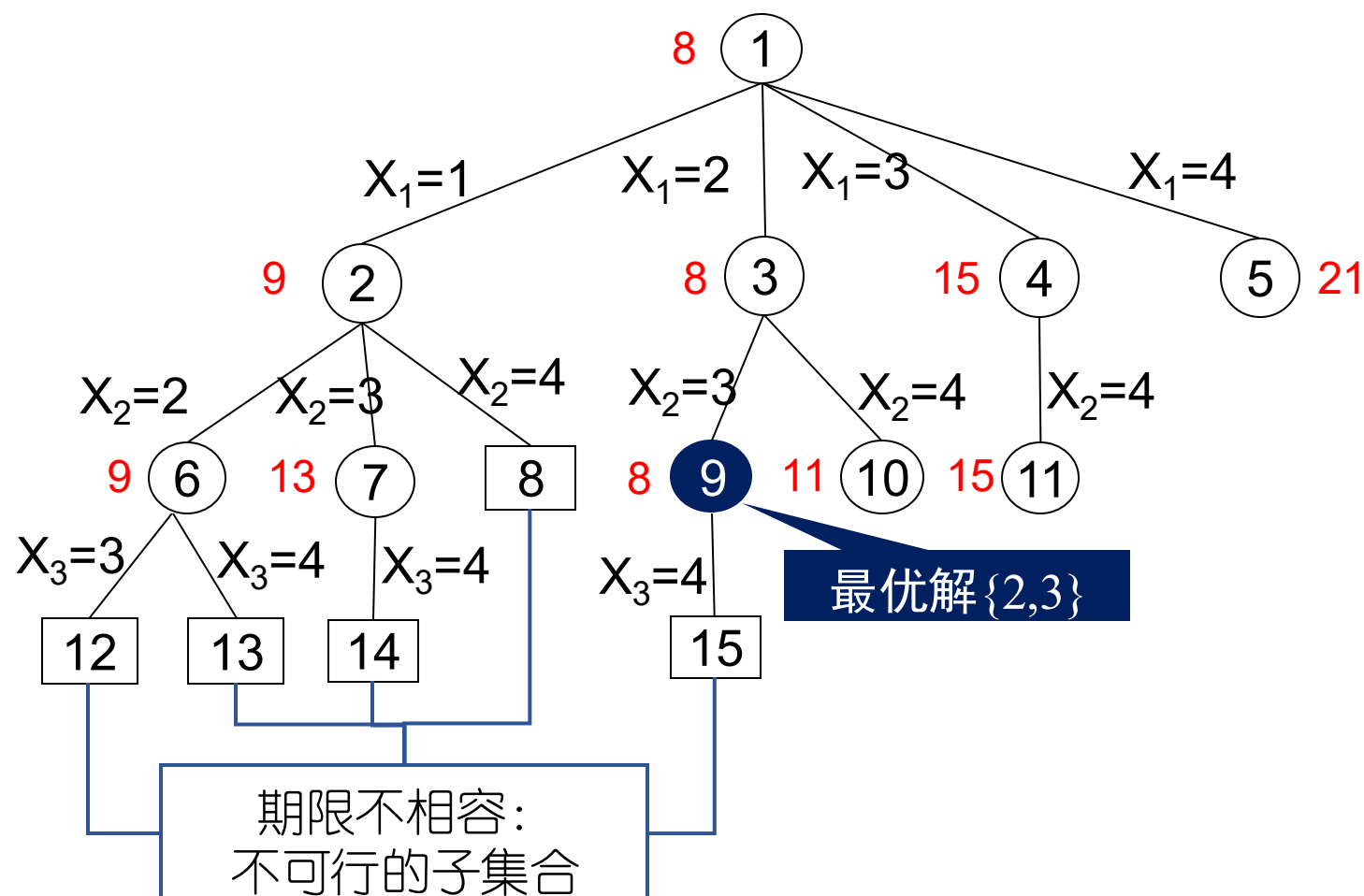
思考: “答案结点 X , 有 $\hat{c}(X) = c(X)$ ” 容易满足吗?

作业	1	2	3	4
p	5	10	6	3
d	1	3	2	1
t	1	2	1	1

- 状态空间树
 - 共计 2^n 个结点
 - 圆形结点表示答案结点(可行解)
 - 方形结点表示不可行结点

如果X是答案结点

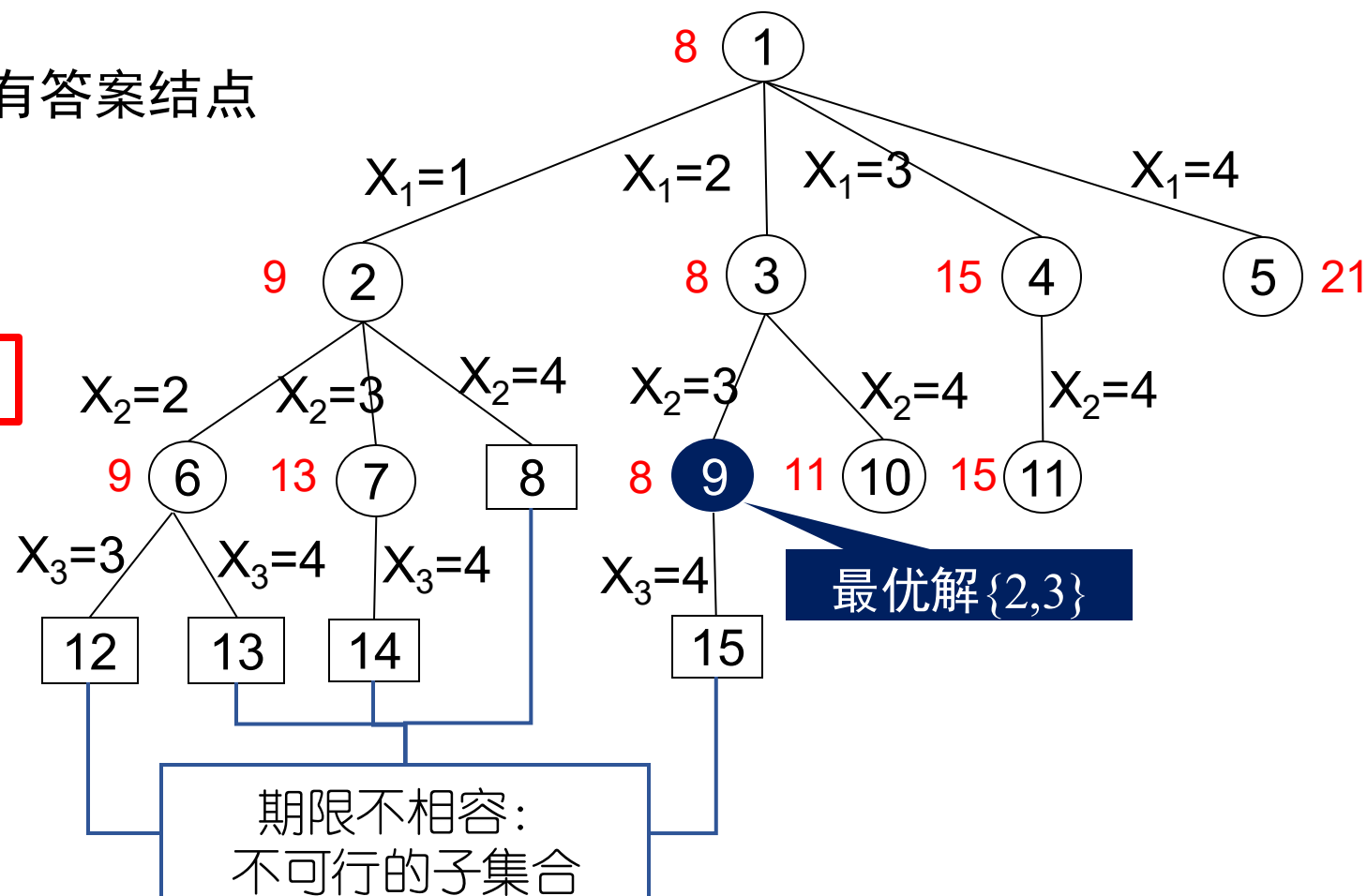
- 若X是叶子结点, $c(X)=X$ 的目标函数值
- 否则, $c(X)=$ 子树X中具有最小成本的答案结点的成本



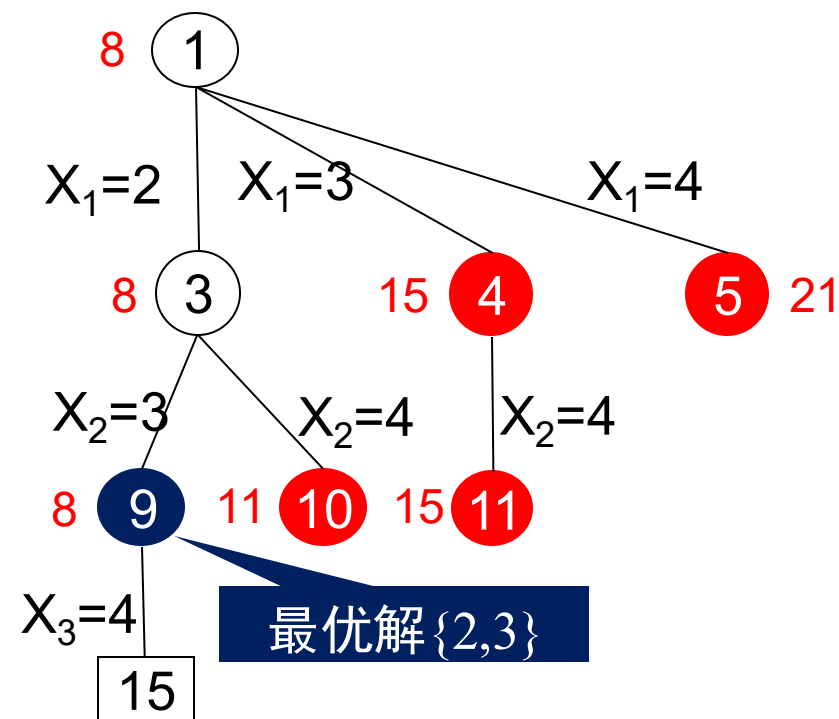
思考: “答案结点X, 有 $\hat{c}(X) = c(X)$ ” 容易满足吗?

- 一种可行方法
 - 使用FIFO, LIFO, LC检索出所有答案结点
 - 找出最小成本的答案结点

思考：是否能进一步优化？



- 一种可行方法
 - 使用FIFO, LIFO, LC检索出所有答案结点
 - 找出最小成本的答案结点
- 若目前检索到的最优解是8
- 还需继续检索10, 11, 4, 5吗?
- 在检索过程中, 剪枝不可能得到更优解的结点!
- 分支-限界法!



思考：如何得知结点不可能得到更优的解？

- 设计最小成本的上界U
- U的取值：
 - 初始值 $\geq$ 最小成本答案结点的成本，通过启发性方法得到，或 ∞
 - 随着结点的访问不断改小
- U的使用：
 - $\hat{c}(X) > U$ 的所有活结点X可以被杀死；
 - 若U来自于一个答案结点, $\hat{c}(X) \geq U$ 的所有活结点X可以被杀死

- $u(X)$ 为结点X的上界估计函数: $c(X) \leq u(X)$
- 使用上界估计函数更新U: 每次添加活结点X时, $U \leftarrow \min(U, u(X))$

思考: 如何判断U是否来自一个答案结点?

- $u(X)$ 为结点X的上界估计函数: $c(X) \leq u(X)$
- 找到一个足够小的正常数 ε
 - 满足对于任意结点: 如果 $u(X) < u(Y)$, $u(X) + \varepsilon < u(Y)$
- 每次添加活结点X时
 - 如果X是答案结点, $U \leftarrow \min(U, X \text{的目标函数值}, u(X) + \varepsilon)$
 - 否则, $U \leftarrow \min(U, u(X) + \varepsilon)$
- $\hat{c}(X) \geq U$ 的所有活结点X可以被杀死

$u(X) + \varepsilon$ 有可能 $< X$ 的目标函数值, 常见于可变k-元组状态空间树

- 步骤1：初始化

- 令根结点T是当前E-结点

- 对U和最优解ans赋初值：

- 如果T是答案结点， $U \leftarrow \min(\text{T的目标函数值}, u(T) + \varepsilon)$ ， $\text{ans} \leftarrow T$

- 否则， $U \leftarrow u(T) + \varepsilon$ ， $\text{ans} \leftarrow 0$

- 活结点队列置为空

- 步骤2：对E的每个子结点X进行检验，添加到活结点队列中，并修改U值：

- 如果 $\hat{c}(X) < U$ ，将X加入到活结点队列中

忽略 $\hat{c}(X) \geq U$ 的结点

- 更新U：

- 如果X是答案结点且目标函数值 $< U$ ， $U \leftarrow \min(\text{X的目标函数值}, u(X) + \varepsilon)$ ， $\text{ans} \leftarrow X$

- 否则，如果 $u(X) + \varepsilon < U$ ， $U \leftarrow u(X) + \varepsilon$

- 步骤3：若活结点队列为空或者下一个E-结点有 $\hat{c} \geq U$ ，打印当前ans；否则从活结点队列中获取下一个活结点E，转到步骤2

$c(X) \geq \hat{c}(X) \geq U \geq \text{最优值}$
无需继续检索

假定状态空间树T至少包含一个答案结点，不可行结点的估计值 $\hat{c}(X) = \infty$ ；可行结点的估计值 $\hat{c}(X) \leq c(X) \leq u(X)$

$\text{cost}(X)$: X的目标函数值

```
procedure LCBB( $T, \hat{c}, u, \varepsilon, \text{cost}$ )
```

```
   $E \leftarrow T$ ;  $\text{PARENT}(E) \leftarrow 0$ 
```

```
  if T是答案结点 then  $U \leftarrow \min(\text{cost}(T), u(T) + \varepsilon)$ ,  $\text{ans} \leftarrow T$ ; else  $U \leftarrow u(T) + \varepsilon$ ,  $\text{ans} \leftarrow 0$  end
```

```
  将活结点表初始化为空
```

初始化

```
  loop
```

```
    for E的每个儿子X do
```

```
      if  $\hat{c}(X) < U$  then call  $\text{ADD}(X)$ ;  $\text{PARENT}(X) \leftarrow E$  end
```

```
      if X是答案结点 and  $\text{cost}(X) < U$  then  $U \leftarrow \min(\text{cost}(X), u(X) + \varepsilon)$ ,  $\text{ans} \leftarrow X$  end
```

扩展活结点

```
      if case  $u(X) + \varepsilon < U$  then  $U \leftarrow u(X) + \varepsilon$  end
```

```
    end
```

```
  end
```

```
  if 不再有活结点 or 下一个E-结点有 $\hat{c} \geq U$  then
```

```
    print (U); 输出从ans到T的那条路径; return
```

```
  end
```

```
  LEAST(E);
```

判断最优解
得到E-节点

```
end
```

```
end
```

思考：如何修改为FIFO分支-限界算法？

成本估计函数 $\hat{c}$ 需满足的性质

- 易于计算
- ~~对于答案结点 X , 有 $\hat{c}(X) = c(X)$~~
- 对于每一个结点 X , $\hat{c}(X) \leq c(X)$

可变 k -元组状态空间树中很难满足

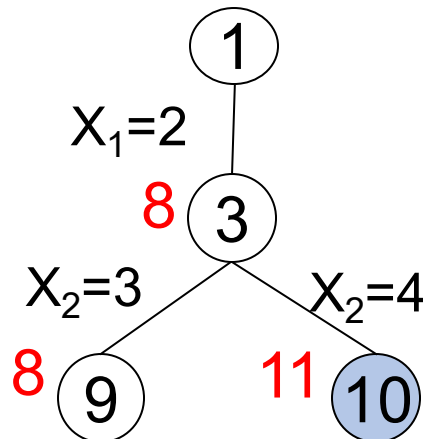
分枝-限界法保证能找到最小成本的答案结点！

带期限的作业调度问题中的成本下界函数 $\hat{c}$

- 定义下界函数 $\hat{c}(X)$ ，使得 $\hat{c}(X) \leq c(X)$
- 设 S_X 是根结点到达结点 X 时选中的作业集合
- $c(X)$ 表示在选择 S_X 的情况下，最少的罚款总数
- 假设 X 后续的作业都不超时，则得到一个下界
- 若 X 是可行解， $\hat{c}(X) = \sum_{i < X, i \notin S_X} p_i$
- 若 X 不是可行解， $\hat{c}(X) = \infty$

作业	1	2	3	4
p	5	10	6	3
d	1	3	2	1
t	1	2	1	1

$$c(1)=8$$



$$\hat{c}(1)=0$$

$$\hat{c}(3)=5 \{1\}$$

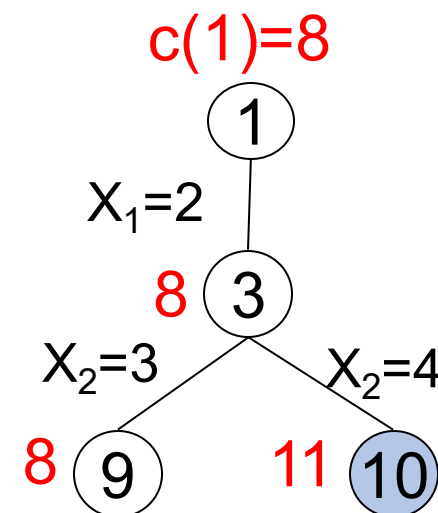
$$\hat{c}(9)=5 \{1\}$$

$$\hat{c}(10)=11 \{1,3\}$$

对于答案结点 X ，当是叶结点时， $\hat{c}(X)=c(X)$ ；
否则， $\hat{c}(X) \neq c(X)$

- 定义上界函数 $u(X)$ ，使得 $c(X) \leq u(X)$
- 设 S_X 是根结点到达结点X时选中的作业集合
- $c(X)$ 表示在选择 S_X 的情况下，最少的罚款总数
- 假设X后续的作业都超时，则得到一个上界
- $u(X) = \sum_{i \notin S_X} p_i$

作业	1	2	3	4
p	5	10	6	3
d	1	3	2	1
t	1	2	1	1



$$u(1)=24 \quad \{1,2,3,4\}$$

$$u(3)=14 \quad \{1,3,4\}$$

$$u(9)=8 \quad \{1,4\}$$

$$u(10)=11 \quad \{1,3\}$$

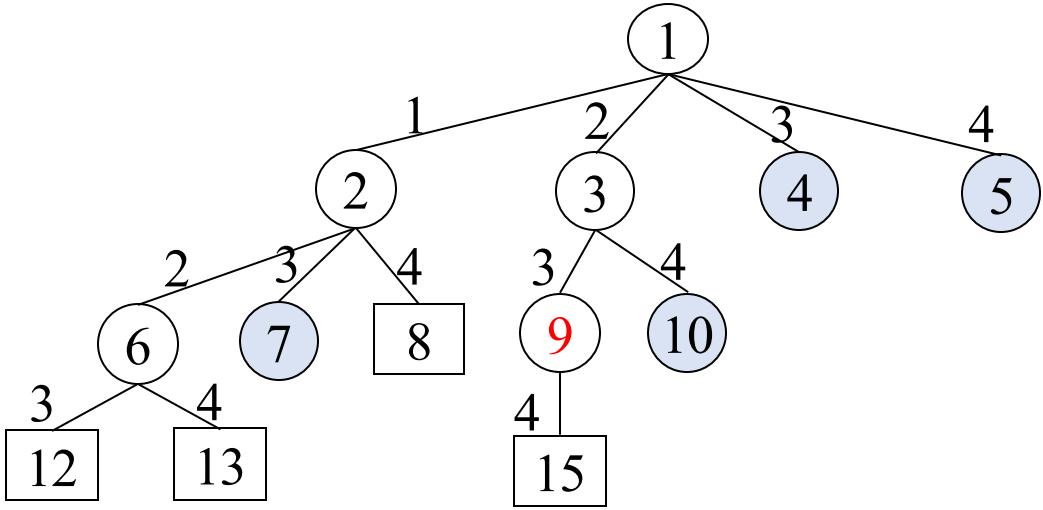
FIFO分支-限界法运行实例

作业	1	2	3	4
p	5	10	6	3
d	1	3	2	1
t	1	1	1	1

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\hat{c}$	0	0	5	15	21	0	10	∞	5	11	∞	∞	∞	∞	∞
cost	24	19	14	18	21	9	13	∞	8	11	∞	∞	∞	∞	∞
u	24	19	14	18	21	9	13	∞	8	11	∞	∞	∞	∞	∞

$\varepsilon=0.1$

U	ans	E
24	1	1
19	2	
14	3	2
9	6	3
8	9	6
		9



队列 2 3 6 9

- $u(X)$ 为结点 X 的上界估计函数: $c(X) \leq u(X)$
- 固定 $\langle X_1, X_2, \dots, X_k \rangle$, 无论 $X_{k+1}, \dots, X_n$ 取何值, 所能获得的最大效益值的下界
(最小目标函数值的上界)

$$\min_{k < i \leq n} -p_i X_i, \quad s.t.,$$

$$\sum_{k < i \leq n} w_i X_i \leq M - \sum_{1 \leq i \leq k} w_i X_i, X_i = 0, 1$$

0/1背包问题
贪心法总能给出效益值的一个下界

Procedure UB (p, w, k, M)

// 物品已按照单位价值进行非升序排列

// p : 前 k 个决策对应获得的价值; w : 前 k 个决策对应消耗的重量

// 基于贪心方法考虑物品 $k+1$ 到 n , 在 $M-w$ 约束条件下的贪心解

$pp \leftarrow p$; $ww \leftarrow w$

 for $i \leftarrow k+1$ to n do

 if $ww + W(i) \leq M$ then $ww \leftarrow ww + W(i)$; $pp \leftarrow pp + P(i)$

 else return $-pp$ end

 end

 return $-pp$

end

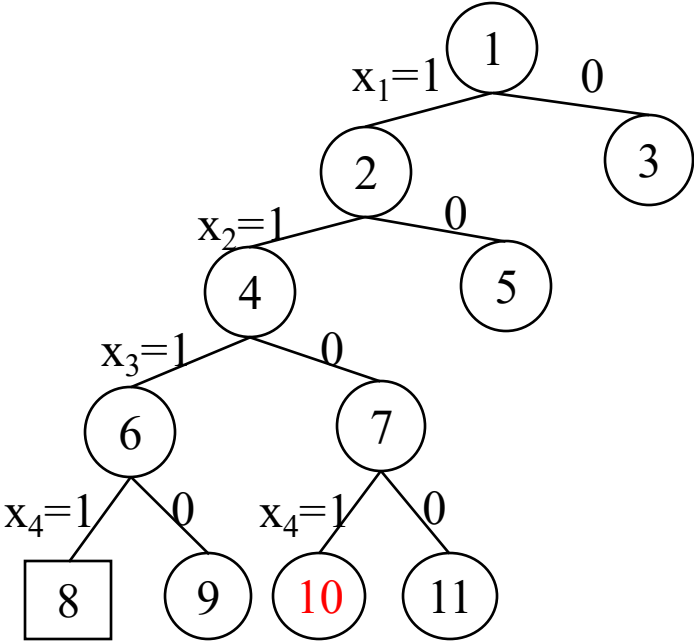
LC分枝-限界法运行实例

- $n=4, M=15, (p_1,p_2,p_3,p_4)=(10,10,12,18), (w_1,w_2,w_3,w_4)=(2,4,6,9)$
- $\varepsilon=0.1$

结点编号	1	2	3	4	5	6	7	8	9	10	11
$\hat{c}$	-38	-38	-32	-38	-36	-38	-38	∞	-32	-38	-20
u	-32	-32	-22	-32	-22	-32	-38	∞	-32	-38	-20

活结点表	U	ans	E	儿子结点
空	-31.9		1	2,3入表
2,3	-31.9		2	4,5入表
3,4,5	-31.9		4	6,7入表
3,5,6,7	-37.9		6	8和9的 $\hat{c}$ 大于U
3,5,7	-37.9		7	10入表, 11的 $\hat{c}$ 大于U
3,5,10	-38	10	10	

$\hat{c} \geq U$, 算法结束

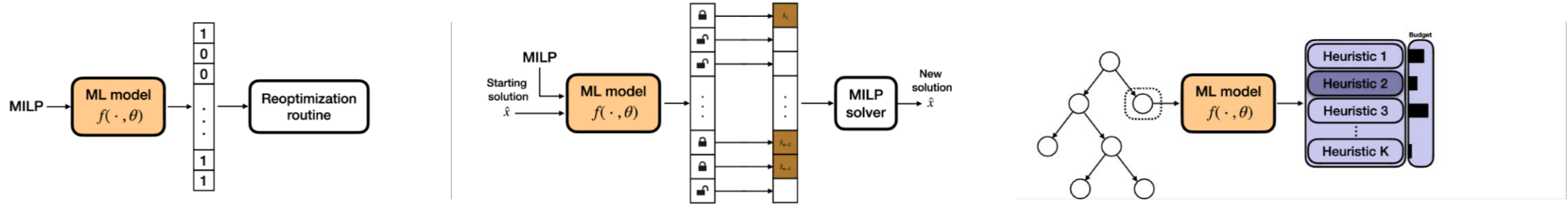


- 8.1 分支-限界法之检索方法
- 8.2 LC-检索
- 8.3 15-谜问题
- 8.4 再论LC-检索
- 8.5 0/1背包问题
- 8.6 分支-限界法之限界方法
- 8.7 小结

- 与回溯法的区别：主要用来求解组合优化问题（寻找目标函数最优的解）
- 目标函数 $\rightarrow$ 成本函数 $c(X)$
- 分支：
 - FIFO、LIFO
 - LC-检索：利用成本下界 $\hat{c}(X)$ 智能分支
 - 易于计算
 - 对于答案结点 X ，有 $\hat{c}(X) = c(X)$
 - 对于每一个结点 X ， $\hat{c}(X) \leq c(X)$
- 限界：利用成本上界 $U \geq c(X)$ 动态限界（剪枝）
 - U 的更新方式
 - 忽略（剪枝）所有 $\hat{c}(X) \geq U$ 的结点

高阶技巧：

- 智能搜索策略
 - 启发式（强分支等）
 - 元算法（遗传算法、模拟退火等）
 - 强化学习
- 任务并行
 - 多核与 GPU 加速
 - 动态任务调度
- 挖掘问题结构（如对称性等）



- Awesome ML4CO Repo: <https://github.com/Thinklab-SJTU/awesome-ml4co>
- IJCAI20 ML4CO Tutorial: <https://sites.google.com/view/ml4co-ijcai20/home>
- NeurIPS21 ML4CO Competition: <https://neurips.cc/virtual/2021/competition/32547>